

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

HỆ THỐNG VOICEBOT TƯ VẤN DINH DƯỠNG THỜI GIAN THỰC DỰA TRÊN AI HỘI THOẠI

TRẦN QUANG MINH
MSSV: 20225047

Chương trình đào tạo: Khoa học máy tính

Giảng viên hướng dẫn: Nguyễn Thị Thu Trang

Khoa: Khoa học máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

TÓM TẮT NỘI DUNG ĐỒ ÁN

Tư vấn dinh dưỡng qua giọng nói là một bài toán có ý nghĩa thực tiễn trong các hệ thống chăm sóc sức khỏe thông minh, tổng đài tự động và trợ lý cá nhân. Người dùng có thể đặt câu hỏi bằng ngôn ngữ tự nhiên về chế độ ăn, vi chất, nhóm thực phẩm hoặc tình trạng sức khỏe thường gặp, sau đó nhận lại câu trả lời dưới dạng giọng nói. Tuy nhiên, bài toán này đặt ra nhiều thách thức hơn so với chatbot văn bản thông thường. Hệ thống cần nhận dạng chính xác giọng nói tiếng Việt, truy xuất được tri thức dinh dưỡng phù hợp, sinh câu trả lời ngắn gọn, có căn cứ và tổng hợp âm thanh với độ trễ đủ thấp để hội thoại không bị ngắt quãng. Đặc biệt, do chủ đề dinh dưỡng liên quan đến sức khỏe, hệ thống cần tránh trả lời ngoài phạm vi, hạn chế suy diễn thiếu bằng chứng và không đưa ra khuyến nghị y tế quá mức.

Đồ án này xây dựng một hệ thống voicebot tư vấn dinh dưỡng tiếng Việt theo kiến trúc pipeline mô-đun gồm Gateway, ASR, mô-đun xử lý hội thoại và TTS. ASR chuyển tín hiệu giọng nói thành transcript; mô-đun xử lý hội thoại thực hiện guardrail đầu vào, chuẩn hóa truy vấn, query expansion, truy xuất tài liệu từ Qdrant, reranking và sinh câu trả lời dựa trên Retrieval-Augmented Generation; TTS chuyển câu trả lời thành giọng nói và stream âm thanh về trình duyệt. Điểm đóng góp chính của đồ án là tổ chức pipeline Voice RAG dưới ràng buộc độ trễ cho chủ đề dinh dưỡng, trong đó hệ thống chuẩn hóa đầu vào sau ASR, truy xuất nhiều chunk ứng viên, chọn các chunk phù hợp nhất bằng reranking và kiểm soát mức độ bằng chứng trước khi sinh câu trả lời. Cách tổ chức này giúp hệ thống không chỉ trả lời nhanh mà còn có cơ chế kiểm soát độ tin cậy của nội dung trong hội thoại.

Kết quả thực nghiệm tập trung vào độ trễ phản hồi, đặc biệt là Time-

to-First-Audio, tức thời gian từ khi có câu hỏi đến khi người dùng nghe được âm thanh đầu tiên. Thay vì chờ LLM sinh xong toàn bộ câu trả lời rồi mới gọi TTS, hệ thống sử dụng cơ chế streaming song song giữa LLM và TTS: văn bản được gom theo ngưỡng ký tự và điểm ngắt câu, sau đó gửi từng đoạn sang TTS ngay khi đủ điều kiện.

Sinh viên thực hiện

Trần Quang Minh

MỤC LỤC

TÓM TẮT NỘI DUNG ĐỒ ÁN	i
1 GIỚI THIỆU BÀI TOÁN	1
1.1 Đặt vấn đề	1
1.2 Mục tiêu và phạm vi	4
1.3 Định hướng giải pháp	6
1.4 Cấu trúc đồ án	7
2 CƠ SỞ LÝ THUYẾT	9
2.1 Nhận dạng giọng nói tự động	9
2.2 Mô hình ngôn ngữ lớn và Retrieval-Augmented Generation	10
2.3 Biểu diễn ngữ nghĩa, vector database và reranking	11
2.4 Đánh giá hệ thống RAG	12
2.5 Tổng hợp giọng nói	13
2.6 Pipeline mô-đun và streaming bất đồng bộ	14
3 GIẢI PHÁP ĐỀ XUẤT	16
3.1 Kiến trúc tổng quan	16
3.1.1 Yêu cầu thiết kế của giải pháp	19
3.1.2 Luồng dữ liệu và trạng thái phiên	19
3.2 Mô-đun xử lý tiếng nói	20
3.2.1 Thiết kế chiều nhận dạng giọng nói	21
3.2.2 Thiết kế chiều tổng hợp giọng nói	21

3.3	Mô-đun xử lý hội thoại	22
3.3.1	Tổ chức ba bước Retrieval, Augmentation và Generation	24
3.3.2	Kiểm soát phạm vi và chất lượng câu trả lời . . .	25
3.4	Tối ưu pipeline Voice RAG dưới ràng buộc độ trễ	25
3.4.1	Phân tích trade-off chất lượng và độ trễ	27
3.4.2	Kiểm soát bằng chứng ở mức thực dụng	28
3.5	Cơ chế streaming song song	28
3.5.1	Mô hình thời gian của pipeline	30
3.5.2	Đảm bảo tính liên mạch của audio	30
3.6	Các tính năng để hệ thống có thể triển khai thực tế	31
3.6.1	Nguyên tắc xử lý lỗi	33
3.6.2	Các giới hạn thiết kế	33
4	PHÁT TRIỂN HỆ THỐNG, THỰC NGHIỆM VÀ ĐÁNH GIÁ	34
4.1	Xây dựng kho tri thức	34
4.2	Phát triển hệ thống	35
4.2.1	Thiết kế hệ thống	35
4.2.2	Xây dựng hệ thống	37
4.3	Thiết lập thực nghiệm	38
4.4	Đánh giá chất lượng RAG	39
4.5	Đánh giá độ trễ	42
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	45
5.1	Kết luận	45

5.2	Hạn chế	46
5.3	Hướng phát triển	47

CHƯƠNG 1. GIỚI THIỆU BÀI TOÁN

1.1 Đặt vấn đề

Hệ thống hội thoại hỏi đáp tư vấn là hệ thống cho phép người dùng đặt câu hỏi bằng ngôn ngữ tự nhiên và nhận lại câu trả lời có tính hỗ trợ ra quyết định trong một phạm vi chuyên môn nhất định. Khác với công cụ tra cứu thông tin đơn thuần, hệ thống hội thoại cần hiểu ý định của câu hỏi, duy trì ngữ cảnh trao đổi, truy xuất hoặc suy luận từ nguồn tri thức phù hợp và diễn đạt câu trả lời theo cách dễ hiểu đối với người dùng cuối. Tùy theo kênh tương tác, hệ thống có thể tồn tại dưới dạng chatbot văn bản hoặc voicebot. Ví dụ, với câu hỏi “người bị tiểu đường nên hạn chế thực phẩm nào”, chatbot văn bản nhận câu hỏi qua bàn phím và trả lời trên màn hình, trong khi voicebot nhận câu hỏi qua giọng nói và phản hồi lại bằng âm thanh.

Các hệ thống hỏi đáp tư vấn có ý nghĩa thực tiễn lớn trong giáo dục, chăm sóc khách hàng, y tế, tài chính cá nhân và chăm sóc sức khỏe. Chúng giúp người dùng tiếp cận thông tin nhanh hơn, giảm tải cho nhân sự tư vấn trực tiếp và hỗ trợ phục vụ ngoài giờ hành chính. Trong bối cảnh các mô hình ngôn ngữ lớn phát triển mạnh, hệ thống hội thoại không chỉ trả lời theo kịch bản cố định mà có thể xử lý nhiều cách diễn đạt tự nhiên hơn, kết hợp với kho tri thức bên ngoài để tạo ra câu trả lời có căn cứ.

Trong lĩnh vực dinh dưỡng, nhu cầu tư vấn thường xuyên có xu hướng tăng nhưng khả năng tiếp cận chuyên gia không phải lúc nào cũng sẵn có. Người dùng phổ thông, đặc biệt là người cao tuổi, người có bệnh nền hoặc người sống xa cơ sở y tế, có thể gặp rào cản khi tìm kiếm thông tin đáng tin cậy và dễ hiểu. Nếu chỉ dựa vào tra cứu thủ công trên Internet, người dùng dễ gặp thông tin rời rạc, khó kiểm chứng hoặc không phù

hợp với tình huống cụ thể. Đây là bối cảnh thúc đẩy nhu cầu xây dựng các hệ thống tư vấn tự động có khả năng hỗ trợ ban đầu, giảm tải cho chuyên gia và giúp người dùng tiếp cận thông tin dinh dưỡng một cách thuận tiện hơn.

Chatbot văn bản là hình thức phổ biến nhất của hệ thống hội thoại hỏi đáp. Ưu điểm của chatbot là dễ triển khai trên website, ứng dụng di động hoặc nền tảng nhắn tin; người dùng có thể đọc lại câu trả lời, sao chép thông tin và gửi câu hỏi chi tiết. Tuy nhiên, kênh văn bản cũng có các hạn chế rõ ràng. Người dùng cần có màn hình, bàn phím hoặc thao tác chạm; việc nhập câu hỏi dài có thể bất tiện; người cao tuổi, người thị lực kém hoặc người đang ở trong tình huống không thể dùng tay sẽ khó tương tác. Vì vậy, chatbot văn bản phù hợp với các tình huống người dùng đang chủ động nhìn màn hình và có đủ thời gian thao tác, nhưng chưa tối ưu cho các ngữ cảnh cần trao đổi nhanh, rảnh tay hoặc gần với hội thoại tự nhiên.

Voicebot ra đời để khắc phục một phần các hạn chế trên bằng cách đưa kênh giọng nói vào quá trình hỏi đáp. Một voicebot thường gồm các thành phần chính: mô-đun nhận dạng giọng nói để chuyển audio thành văn bản; mô-đun xử lý hội thoại để hiểu câu hỏi, truy xuất tri thức và sinh câu trả lời; mô-đun tổng hợp giọng nói để chuyển câu trả lời thành âm thanh; và Gateway để điều phối luồng dữ liệu giữa người dùng với các mô-đun phía sau. Kênh voice đặc biệt hữu ích trong các tổng đài tự động, ứng dụng trợ lý giọng nói, hệ thống chăm sóc khách hàng, môi trường lái xe, hỗ trợ người mắt kém hoặc các tình huống người dùng không thuận tiện thao tác bằng tay. Nhờ đó, voicebot có tiềm năng đưa hệ thống tư vấn đến gần hơn với cách giao tiếp tự nhiên của con người.

Tuy nhiên, kênh voice cũng đặt ra nhiều thách thức hơn so với kênh văn bản. Thứ nhất là độ chính xác: một hệ thống voice-to-voice phải đi qua nhiều bước liên tiếp như nhận dạng giọng nói, hiểu câu hỏi, truy

xuất tri thức, sinh câu trả lời và tổng hợp giọng nói. Sai số ở bước trước có thể lan truyền sang bước sau; ví dụ transcript sai có thể làm truy xuất sai tài liệu, từ đó khiến câu trả lời không đúng trọng tâm. Thứ hai là độ trễ: thay vì chỉ sinh câu trả lời văn bản, hệ thống phải xử lý audio đầu vào, gọi mô hình ngôn ngữ, truy xuất dữ liệu và tổng hợp audio đầu ra. Nếu các bước này chạy tuần tự, tổng thời gian chờ sẽ dài và làm hội thoại bị đứt mạch. Thứ ba là trải nghiệm nghe: câu trả lời cần ngắn gọn, dễ nghe, không chứa định dạng khó đọc và không bị ngắt câu thiếu tự nhiên khi phát âm thanh.

Trong chủ đề dinh dưỡng, nhu cầu tư vấn qua hội thoại càng rõ ràng. Người dùng thường có các câu hỏi thực tế như lựa chọn thực phẩm, chế độ ăn giảm cân, bổ sung vitamin, dinh dưỡng cho phụ nữ mang thai, trẻ biếng ăn, người cao tuổi hoặc người mắc các bệnh nền phổ biến. Đây là nhóm câu hỏi có tần suất cao, liên quan trực tiếp đến sinh hoạt hằng ngày và phù hợp với một hệ thống hỗ trợ tư vấn ban đầu. Tuy vậy, dinh dưỡng cũng là chủ đề liên quan đến sức khỏe nên hệ thống không được trả lời tùy tiện. Câu trả lời cần dựa trên nguồn tri thức đáng tin cậy, tránh khẳng định quá mức, không thay thế chẩn đoán hoặc điều trị y khoa và cần biết từ chối hoặc khuyến nghị gặp chuyên gia trong các trường hợp nhạy cảm.

Từ các vấn đề trên, đề án tập trung xây dựng một hệ thống voicebot tư vấn dinh dưỡng tiếng Việt thời gian thực. Hệ thống giải quyết bài toán tổ chức pipeline ASR–RAG–LLM–TTS để người dùng có thể đặt câu hỏi bằng giọng nói, hệ thống truy xuất tri thức dinh dưỡng phù hợp, sinh câu trả lời có căn cứ và phát phản hồi âm thanh với độ trễ đủ thấp cho hội thoại tự nhiên.

1.2 Mục tiêu và phạm vi

Phạm vi của đề án được giới hạn trong bài toán tư vấn dinh dưỡng bằng tiếng Việt qua kênh thoại. Về ngôn ngữ, hệ thống tập trung xử lý tiếng Việt ở cả đầu vào giọng nói, câu hỏi văn bản trung gian và câu trả lời đầu ra. Về chủ đề, hệ thống chỉ xử lý các câu hỏi dinh dưỡng phổ biến như lựa chọn thực phẩm, chế độ ăn hằng ngày, vi chất, nhóm thực phẩm, kiểm soát cân nặng, dinh dưỡng cho phụ nữ mang thai, trẻ em, người cao tuổi và một số tình trạng sức khỏe thường gặp. Đề án không đặt mục tiêu thay thế bác sĩ, chuyên gia dinh dưỡng hoặc hệ thống chẩn đoán y khoa.

Về kênh tương tác, đề án tập trung vào voicebot thay vì chatbot văn bản thuần túy. Người dùng đặt câu hỏi bằng giọng nói, hệ thống nhận dạng thành văn bản, truy xuất tri thức, sinh câu trả lời và phát lại bằng giọng nói. Giao diện văn bản nếu có chỉ đóng vai trò hỗ trợ hiển thị transcript, trạng thái xử lý hoặc thông tin debug; mục tiêu chính vẫn là trải nghiệm hội thoại thoại thời gian thực.

Về dữ liệu, kho tri thức ban đầu được xây dựng từ 1060 bài viết dinh dưỡng thu thập từ Báo Sức khỏe và Đời sống, trang web của Viện Dinh dưỡng Quốc gia và Bệnh viện Đa khoa Quốc tế Thu Cúc. Các bài viết được làm sạch, chuẩn hóa và phân đoạn thành các chunk phục vụ embedding, truy xuất và reranking. Chất lượng dữ liệu được xem xét theo ba tiêu chí: nguồn thông tin có tính chuyên môn, nội dung phù hợp với chủ đề dinh dưỡng phổ biến và chunk sau xử lý đủ ngắn để truy xuất chính xác nhưng vẫn giữ đủ ngữ cảnh cho LLM sinh câu trả lời.

Các mục tiêu cụ thể của đề án gồm:

- 1) Xây dựng pipeline voice-to-voice gồm ASR, mô-đun xử lý hội thoại và TTS, có khả năng xử lý một lượt hỏi đáp dinh dưỡng từ giọng nói đầu vào đến giọng nói đầu ra.

- 2) Tối ưu độ trễ phản hồi, trong đó chỉ số trọng tâm là Time-to-First-Audio. Hệ thống được đánh giá theo p50, p90 và p99 để phản ánh cả trường hợp trung bình và trường hợp chậm.
- 3) So sánh pipeline tuần tự với pipeline streaming LLM–TTS, đồng thời đánh giá ảnh hưởng của kích thước chunk đến Time-to-First-Audio, tổng thời gian phản hồi và độ tự nhiên của âm thanh.
- 4) Nâng cao độ chính xác nội dung bằng RAG trên kho tri thức dinh dưỡng, kết hợp query expansion, truy xuất vector, reranking và prompt ràng buộc theo ngữ cảnh.
- 5) Đánh giá chất lượng câu trả lời theo các tiêu chí: câu trả lời đúng trọng tâm câu hỏi, bám sát ngữ cảnh truy xuất, hạn chế suy diễn ngoài tài liệu và phù hợp với kênh thoại.
- 6) Thiết kế hệ thống theo pipeline mô-đun để dễ triển khai, kiểm thử và mở rộng, trong đó Gateway có nhiệm vụ điều phối luồng dữ liệu, hàng đợi text–audio và trạng thái phiên hội thoại.

Các câu hỏi đánh giá xoay quanh các chủ đề như đại tháo đường, phụ nữ mang thai, trẻ biếng ăn, vitamin, khoáng chất, chế độ ăn giảm cân, omega-3, canxi và các nhóm thực phẩm thường gặp. Hệ thống không tự đưa ra chẩn đoán bệnh, không kê đơn thuốc và không thay đổi phác đồ điều trị. Với các câu hỏi vượt phạm vi hoặc có dấu hiệu rủi ro y tế, hệ thống cần từ chối mềm, khuyến nghị người dùng gặp chuyên gia phù hợp hoặc yêu cầu đặt lại câu hỏi trong phạm vi dinh dưỡng.

Như vậy, đề án không chỉ đánh giá hệ thống theo khả năng sinh câu trả lời đúng, mà còn theo khả năng vận hành trong tương tác giọng nói thời gian thực. Hai nhóm chỉ số chính là: nhóm độ trễ gồm Time-to-First-Audio, tổng thời gian phản hồi và độ trễ từng mô-đun; nhóm chất lượng gồm mức liên quan của tài liệu truy xuất, độ bám ngữ cảnh của câu trả lời và mức phù hợp của câu trả lời khi chuyển thành giọng nói.

1.3 Định hướng giải pháp

Định hướng giải pháp của đề án là xây dựng một pipeline voice-to-voice cho tư vấn dinh dưỡng tiếng Việt. Khi người dùng đặt câu hỏi bằng giọng nói, hệ thống trước hết chuyển tín hiệu âm thanh thành transcript tiếng Việt, sau đó dùng khối xử lý hội thoại để hiểu câu hỏi, truy xuất tri thức liên quan và sinh câu trả lời. Câu trả lời cuối cùng được chuyển lại thành âm thanh để người dùng có thể nghe trực tiếp. Gateway đóng vai trò điều phối phiên hội thoại, kết nối các bước xử lý và giữ cho luồng dữ liệu giữa trình duyệt với các mô-đun phía sau diễn ra liên tục.

Để hạn chế việc mô hình ngôn ngữ trả lời chỉ dựa trên kiến thức nền, hệ thống được định hướng theo Retrieval-Augmented Generation. Kho bài viết dinh dưỡng sau khi làm sạch được biểu diễn dưới dạng embedding và lưu trong vector database để phục vụ truy xuất ngữ nghĩa. Với mỗi câu hỏi, hệ thống tìm các tài liệu liên quan, lọc lại bằng reranking và đưa phần ngữ cảnh phù hợp vào LLM. Cách tiếp cận này giúp câu trả lời có cơ sở từ kho tri thức của hệ thống, đồng thời vẫn tận dụng khả năng diễn đạt tự nhiên của mô hình ngôn ngữ.

Đối với kênh thoại, độ trễ là vấn đề trung tâm của giải pháp. Nếu hệ thống xử lý tuần tự, người dùng phải chờ nhận dạng giọng nói, truy xuất tài liệu, sinh toàn bộ câu trả lời và tổng hợp xong toàn bộ âm thanh rồi mới nghe được phản hồi. Vì vậy, đề án định hướng tổ chức quá trình trả lời theo streaming: khi LLM sinh được một phần văn bản đủ điều kiện, phần văn bản đó được chuyển sớm sang TTS thay vì chờ toàn bộ câu trả lời hoàn tất. Chiến thuật word-safe chunking được sử dụng để chia văn bản tại các vị trí tương đối tự nhiên, nhằm giảm Time-to-First-Audio nhưng vẫn hạn chế việc TTS đọc bị ngắt nhịp.

Các công cụ, mô hình, framework và thuật toán cụ thể để hiện thực định hướng trên sẽ được làm rõ ở các chương sau. Phần cơ sở lý thuyết

trình bày các nền tảng cần thiết cho nhận dạng giọng nói, mô hình ngôn ngữ, truy xuất tăng cường sinh, truy xuất vector, reranking, tổng hợp giọng nói và streaming. Các chương tiếp theo trình bày cách những thành phần này được kết nối thành hệ thống hoàn chỉnh, cũng như cách đánh giá hệ thống theo chất lượng câu trả lời và độ trễ phản hồi.

1.4 Cấu trúc đồ án

Phần còn lại của báo cáo được tổ chức theo mạch từ bối cảnh, phát biểu bài toán, thách thức kỹ thuật đến giải pháp và đánh giá. Chương 1 giới thiệu bối cảnh thiếu hụt chuyên gia dinh dưỡng, rào cản tiếp cận thông tin của các nhóm người dùng như người cao tuổi, đồng thời phát biểu bài toán xây dựng hệ thống voice-to-voice có khả năng tư vấn dựa trên cơ sở tri thức thay vì chỉ dựa vào kiến thức chung của LLM. Chương này cũng nêu các thách thức chính của bài toán, gồm nút thắt độ trễ khi tích hợp nhiều mô hình AI và nguy cơ hallucination trong tư vấn sức khỏe.

Chương 2 trình bày cơ sở lý thuyết liên quan đến nhận dạng giọng nói, mô hình ngôn ngữ, RAG, vector database, embedding, reranking, RAGAS và tổng hợp giọng nói. Các nội dung này cung cấp nền tảng để giải thích vì sao hệ thống cần kết hợp kho tri thức bên ngoài với LLM, cũng như vì sao độ trễ trở thành vấn đề quan trọng trong bài toán voicebot.

Chương 3 mô tả giải pháp đề xuất, bao gồm kiến trúc tổng quan, các mô-đun xử lý tiếng nói, mô-đun xử lý hội thoại, quy trình RAG hai tầng và chiến thuật streaming. Đây là chương làm rõ cách hệ thống tổ chức pipeline voice-to-voice, cách câu trả lời được sinh dựa trên kho tri thức, và cách streaming song song giữa LLM với TTS được sử dụng để tối ưu Time-to-First-Audio.

Chương 4 trình bày quá trình phát triển hệ thống, xây dựng kho tri thức, thiết kế đánh giá thực nghiệm và phân tích ablation study theo từng thành phần. Các thí nghiệm tập trung vào vai trò của truy xuất, reranking, chunking, streaming và các chỉ số độ trễ end-to-end; đồng thời đánh giá chất lượng câu trả lời bằng các tiêu chí của RAGAS ở mức chấp nhận được cho bài toán tư vấn dinh dưỡng.

Chương 5 tổng kết kết quả đạt được và đối chiếu lại các đóng góp chính của đề án. Các đóng góp này gồm pipeline streaming song song tối ưu hóa thời gian phản hồi âm thanh đầu tiên, quy trình RAG hai tầng cho tiếng Việt chuyên ngành dinh dưỡng, và bộ chỉ số đánh giá ablation study để phân tích vai trò của từng mô-đun. Chương này cũng nêu các hạn chế còn tồn tại và định hướng phát triển tiếp theo.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Nhận dạng giọng nói tự động

Nhận dạng giọng nói tự động là bài toán ánh xạ một chuỗi đặc trưng âm học $X = (x_1, x_2, \dots, x_T)$ thành một chuỗi ký hiệu ngôn ngữ $Y = (y_1, y_2, \dots, y_U)$. Khác với bài toán phân loại thông thường, số khung âm thanh T thường lớn hơn nhiều so với số token văn bản U , đồng thời ranh giới giữa các âm vị hoặc từ không được gán nhãn trực tiếp trong dữ liệu huấn luyện. Vì vậy, một khó khăn cốt lõi của ASR là học quan hệ căn chỉnh giữa tín hiệu liên tục và chuỗi văn bản rời rạc.

Connectionist Temporal Classification (CTC) là một trong những hướng tiếp cận quan trọng cho bài toán chuỗi chưa được căn chỉnh. CTC bổ sung ký hiệu rỗng và tính tổng xác suất trên nhiều đường căn chỉnh khả dĩ giữa chuỗi đầu vào và chuỗi đầu ra [1]. Cách tiếp cận này giúp huấn luyện mô hình ASR mà không cần nhãn căn chỉnh ở cấp khung âm thanh. Tuy nhiên, CTC thường giả định tính độc lập có điều kiện giữa các bước dự đoán, do đó kết quả có thể cần thêm mô hình ngôn ngữ hoặc hậu xử lý để cải thiện độ trôi chảy.

Recurrent Neural Network Transducer (RNN-T) mở rộng hướng tiếp cận chuỗi sang chuỗi bằng cách kết hợp ba thành phần: bộ mã hóa âm học, mạng dự đoán dựa trên các token đã sinh và mạng kết hợp để tạo phân phối token tiếp theo [2]. So với các mô hình cần quan sát toàn bộ utterance, RNN-T phù hợp hơn với nhận dạng thời gian thực vì mô hình có thể xử lý dần luồng âm thanh và sinh kết quả từng phần. Đặc điểm này quan trọng đối với voicebot, nơi độ trễ cảm nhận của người dùng phụ thuộc vào khả năng phát hiện kết thúc lời nói và chuyển nhanh transcript sang bước xử lý hội thoại.

Độ chính xác của ASR thường được đo bằng Word Error Rate (WER)

hoặc Character Error Rate (CER), trong đó lỗi thay thế, chèn và xóa được chuẩn hóa theo độ dài câu tham chiếu. Bên cạnh độ chính xác, hệ thống hội thoại còn cần quan tâm đến Real-Time Factor (RTF), tức tỉ lệ giữa thời gian xử lý và thời lượng âm thanh đầu vào. Với pipeline voice-to-voice, ASR không chỉ là một mô-đun độc lập mà còn là nguồn đầu vào cho truy xuất tri thức; lỗi nhận dạng như nhầm thuật ngữ dinh dưỡng, số đo hoặc tên vi chất có thể làm lệch truy vấn và kéo theo lỗi ở các bước phía sau.

2.2 Mô hình ngôn ngữ lớn và Retrieval-Augmented Generation

Transformer là kiến trúc nền tảng của nhiều mô hình ngôn ngữ hiện đại. Khác với các kiến trúc tuần tự truyền thống, Transformer sử dụng cơ chế self-attention để mô hình hóa quan hệ giữa các token trong cùng một chuỗi, từ đó cho phép học biểu diễn ngữ cảnh linh hoạt và song song hóa quá trình huấn luyện hiệu quả hơn [3]. Các mô hình ngôn ngữ lớn xây dựng trên kiến trúc này có khả năng sinh văn bản tự nhiên, tổng hợp thông tin và trả lời câu hỏi theo ngữ cảnh.

Tuy nhiên, trong các bài toán liên quan đến sức khỏe, việc chỉ dựa vào tri thức tham số của LLM là chưa đủ an toàn. Mô hình có thể sinh câu trả lời hợp lý về mặt ngôn ngữ nhưng thiếu căn cứ, không cập nhật hoặc vượt quá phạm vi dữ liệu đáng tin cậy. Retrieval-Augmented Generation (RAG) giải quyết hạn chế này bằng cách kết hợp mô hình sinh với một bộ nhớ ngoài dưới dạng kho tài liệu [4]. Thay vì để LLM trả lời trực tiếp, hệ thống trước hết truy xuất các đoạn văn bản liên quan, đưa chúng vào ngữ cảnh, sau đó yêu cầu LLM sinh câu trả lời dựa trên bằng chứng đã truy xuất.

Một pipeline RAG điển hình gồm bốn giai đoạn: xây dựng chỉ mục tài liệu, truy xuất ngữ cảnh theo câu hỏi, bổ sung ngữ cảnh vào đầu vào

của LLM và sinh câu trả lời. Với bài toán tư vấn dinh dưỡng qua giọng nói, RAG có hai vai trò chính. Thứ nhất, nó giúp câu trả lời bám vào kho tri thức đã kiểm soát thay vì dựa hoàn toàn vào kiến thức chung của mô hình. Thứ hai, nó tạo điều kiện để đánh giá riêng chất lượng truy xuất và chất lượng sinh câu trả lời, qua đó phát hiện hệ thống sai do thiếu tài liệu, chọn sai tài liệu hay do LLM diễn giải chưa đúng.

Trong kênh thoại, câu trả lời của LLM còn chịu thêm ràng buộc về độ dài, độ dễ nghe và độ an toàn. Một câu trả lời quá dài làm tăng thời gian tổng hợp giọng nói và khiến người dùng khó theo dõi. Ngược lại, câu trả lời quá ngắn có thể thiếu điều kiện sử dụng hoặc thiếu cảnh báo cần thiết. Vì vậy, prompt trong hệ thống voicebot cần được thiết kế để vừa yêu cầu tính căn cứ theo ngữ cảnh truy xuất, vừa kiểm soát phong cách trả lời phù hợp với hội thoại nói.

2.3 Biểu diễn ngữ nghĩa, vector database và reranking

Để truy xuất tài liệu theo ngữ nghĩa, các đoạn văn bản trong kho tri thức được biến đổi thành vector embedding. Biểu diễn embedding cho phép các câu có cách diễn đạt khác nhau nhưng cùng ý nghĩa nằm gần nhau trong không gian vector. Các mô hình bi-encoder như Sentence-BERT mã hóa truy vấn và tài liệu độc lập, nhờ đó có thể tính toán vector tài liệu và tìm kiếm nhanh bằng độ tương đồng vector [5]. Đây là cơ sở quan trọng cho semantic search trong các hệ thống RAG.

Khi số lượng vector lớn, việc so sánh truy vấn với toàn bộ kho dữ liệu theo cách tuyến tính sẽ tốn kém. Các thuật toán approximate nearest neighbor được sử dụng để tăng tốc tìm kiếm lân cận trong không gian nhiều chiều. HNSW xây dựng đồ thị nhiều tầng, trong đó quá trình tìm kiếm bắt đầu từ tầng thưa hơn rồi dần đi xuống các tầng dày hơn để tìm các điểm gần truy vấn [6]. Qdrant là vector database hỗ trợ lưu trữ vector,

payload metadata và chỉ mục phục vụ tìm kiếm vector kết hợp với lọc dữ liệu; tài liệu chính thức của Qdrant cũng nhấn mạnh vai trò kết hợp giữa vector index và payload index để tối ưu truy vấn [7].

Bi-encoder có ưu điểm tốc độ nhưng chưa luôn đủ chính xác ở các câu hỏi chuyên ngành, vì nó mã hóa truy vấn và tài liệu độc lập. Cross-encoder khắc phục hạn chế này bằng cách đưa cặp truy vấn–tài liệu vào cùng một mô hình để tính điểm liên quan, nhờ đó khai thác tương tác token chi tiết hơn. Chi phí tính toán của cross-encoder cao hơn, nên thường được dùng ở bước reranking sau khi bi-encoder đã lấy một tập ứng viên ban đầu. Trong hệ thống của đồ án, truy xuất được tổ chức theo hai bước: trước hết hệ thống lấy 15 chunk ứng viên từ vector database, sau đó reranker chọn 5 chunk phù hợp nhất làm ngữ cảnh đầu vào cho LLM. Cách tổ chức này cân bằng giữa độ phủ truy xuất và độ trễ xử lý.

2.4 Đánh giá hệ thống RAG

Đánh giá RAG không thể chỉ dựa trên việc câu trả lời nghe có vẻ tự nhiên. Một câu trả lời có thể trôi chảy nhưng không bám vào tài liệu, hoặc tài liệu truy xuất có thể liên quan nhưng LLM lại sử dụng sai. Vì vậy, cần tách việc đánh giá thành nhiều chiều: chất lượng truy xuất, mức độ tận dụng ngữ cảnh và chất lượng câu trả lời cuối cùng.

RAGAS là một framework đánh giá RAG tập trung vào các tiêu chí như faithfulness, answer relevancy, context precision và context recall [8]. Faithfulness đo mức độ câu trả lời được suy ra từ ngữ cảnh; answer relevancy đo mức độ câu trả lời tập trung vào câu hỏi; context precision phản ánh tỉ lệ ngữ cảnh được chọn là hữu ích; context recall phản ánh khả năng truy xuất đủ thông tin cần thiết. Với chủ đề dinh dưỡng, faithfulness và context recall đặc biệt quan trọng vì câu trả lời cần có bằng chứng và không nên suy diễn vượt quá tài liệu.

Ngoài chất lượng nội dung, voicebot còn cần đánh giá độ trễ. Các chỉ số như Time-to-First-Audio, tổng thời gian phản hồi và RTF của từng mô-đun cho phép phân tích điểm nghẽn trong pipeline. Trong một hệ thống nhiều bước, tổng độ trễ không chỉ phụ thuộc vào từng mô hình riêng lẻ mà còn phụ thuộc vào cách tổ chức luồng dữ liệu giữa các mô-đun. Do đó, đánh giá hệ thống cần kết hợp cả thước đo chất lượng RAG và thước đo trải nghiệm hội thoại.

2.5 Tổng hợp giọng nói

Tổng hợp giọng nói là bài toán chuyển văn bản thành tín hiệu âm thanh có nội dung, ngữ điệu và nhịp điệu phù hợp. Các hệ thống TTS truyền thống thường tách thành nhiều bước như dự đoán đặc trưng âm học và vocoder, trong khi các mô hình neural hiện đại hướng tới học trực tiếp hơn quan hệ giữa văn bản và waveform. VITS là một ví dụ tiêu biểu của mô hình end-to-end kết hợp variational inference, normalizing flow và adversarial learning để sinh giọng nói tự nhiên [9].

Một hướng phát triển khác là neural codec language model, trong đó âm thanh được mã hóa thành các token rời rạc bởi codec âm thanh, sau đó mô hình ngôn ngữ học cách sinh chuỗi token âm thanh có điều kiện theo văn bản hoặc prompt giọng nói. VALL-E minh họa hướng tiếp cận này khi xem TTS như một bài toán mô hình hóa ngôn ngữ trên mã âm thanh rời rạc [10]. Dù kiến trúc cụ thể giữa các hệ thống TTS khác nhau, điểm chung trong bài toán voicebot là mô hình phải cân bằng giữa chất lượng giọng nói, tốc độ sinh âm thanh và tính ổn định khi xử lý nhiều đoạn văn bản ngắn.

Với tiếng Việt, TTS cần xử lý thanh điệu, cách đọc số, tên chất dinh dưỡng, ký hiệu đo lường và ngắt nghỉ câu. Trong hệ thống hội thoại, văn bản đầu vào cho TTS thường được sinh dần từ LLM, do đó cần có chiến

thuật chia đoạn phù hợp. Nếu đoạn quá dài, thời gian chờ âm thanh đầu tiên tăng; nếu đoạn quá ngắn, giọng nói có thể bị ngắt nhịp hoặc thiếu tự nhiên. Vì vậy, đánh giá TTS trong đồ án không chỉ xét chất lượng âm thanh, mà còn xét thời gian tạo âm thanh đầu tiên và ảnh hưởng của kích thước đoạn văn bản đến trải nghiệm nghe.

2.6 Pipeline mô-đun và streaming bất đồng bộ

Voicebot tư vấn dinh dưỡng là một pipeline nhiều bước gồm nhận dạng giọng nói, xử lý hội thoại, truy xuất tri thức, sinh câu trả lời và tổng hợp giọng nói. Nếu các bước này được thực hiện tuần tự, người dùng phải chờ tổng thời gian của toàn bộ pipeline trước khi nghe phản hồi. Với hội thoại tự nhiên, cách tổ chức này dễ tạo cảm giác chậm, đặc biệt khi LLM và TTS đều có chi phí xử lý đáng kể.

Streaming bất đồng bộ giải quyết vấn đề này bằng cách cho phép các mô-đun trao đổi dữ liệu từng phần. Khi LLM sinh được một đoạn văn bản đủ điều kiện, đoạn này có thể được chuyển sang TTS trong khi LLM tiếp tục sinh phần còn lại. Cách chồng lấp thời gian xử lý này không nhất thiết làm giảm tổng chi phí tính toán của từng mô hình, nhưng có thể giảm mạnh Time-to-First-Audio, tức thời điểm người dùng bắt đầu nghe thấy phản hồi. Đây là chỉ số quan trọng hơn tổng thời gian sinh toàn bộ câu trả lời trong trải nghiệm hội thoại.

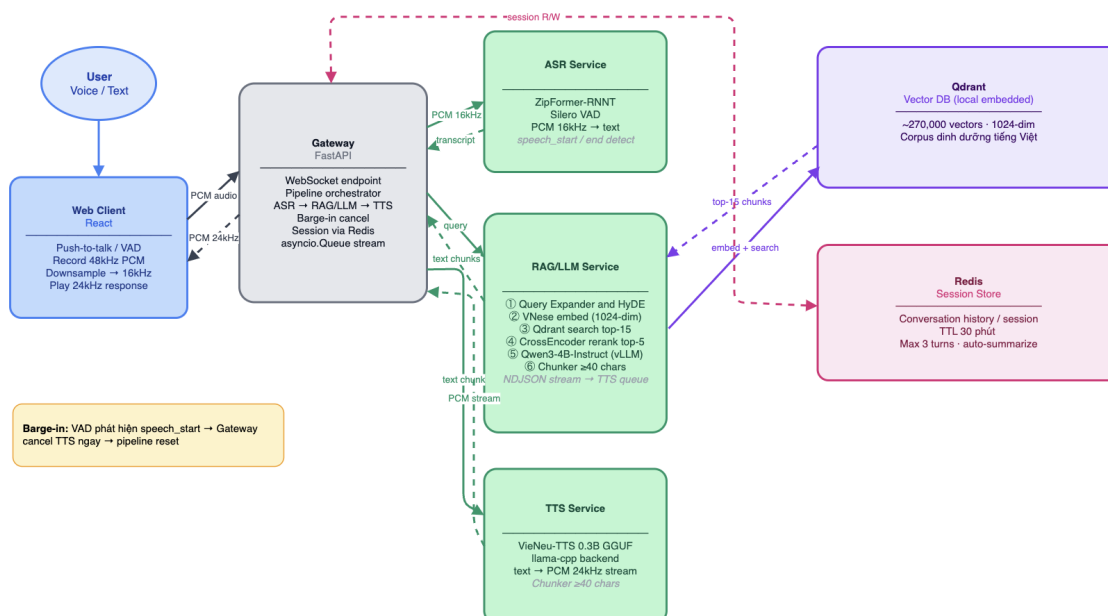
Về mặt triển khai, WebSocket là cơ chế phù hợp cho các luồng dữ liệu hai chiều và kéo dài, vì client và server có thể liên tục gửi nhận audio, transcript, text chunk và audio chunk trong cùng một phiên kết nối. FastAPI hỗ trợ xây dựng các kết nối WebSocket bất đồng bộ trên nền ASGI [11]. Trong phạm vi đồ án, pipeline được mô tả là pipeline mô-đun theo hướng service-oriented: các thành phần ASR, mô-đun xử lý hội thoại và TTS được tách riêng để dễ thay thế, đo đạc và tối ưu, nhưng

vấn phục vụ một luồng nghiệp vụ thống nhất là tư vấn dinh dưỡng qua giọng nói.

CHƯƠNG 3. GIẢI PHÁP ĐỀ XUẤT

3.1 Kiến trúc tổng quan

Hệ thống được thiết kế theo kiến trúc pipeline mô-đun cho bài toán voice-to-voice. Thay vì để một tiến trình xử lý toàn bộ luồng hội thoại, hệ thống tách các chức năng chính thành các khối độc lập gồm Web Client, Gateway, ASR Service, LLM/RAG Service, TTS Service, Qdrant và Redis. Cách tổ chức này giúp từng thành phần có trách nhiệm rõ ràng: Web Client thu và phát audio, Gateway điều phối phiên hội thoại, ASR chuyển giọng nói thành transcript, LLM/RAG truy xuất tri thức và sinh câu trả lời, TTS chuyển văn bản thành âm thanh, Qdrant lưu chỉ mục vector của kho tri thức, còn Redis lưu trạng thái hội thoại theo session.



Hình 3.1: Kiến trúc tổng quan của hệ thống

Luồng xử lý bắt đầu từ Web Client. Người dùng có thể nhập câu hỏi bằng văn bản hoặc nói trực tiếp qua micro. Với kênh thoại, client thu audio PCM, thực hiện một số bước xử lý ở phía trình duyệt như push-to-talk, VAD, ghi âm ở tần số lấy mẫu cao và chuyển đổi về 16 kHz trước khi gửi về Gateway. Gateway tiếp nhận audio, quản lý session và chuyển

audio sang ASR Service để nhận transcript cuối cùng. Transcript sau đó được gửi sang LLM/RAG Service để thực hiện mở rộng truy vấn, HyDE, embedding, truy xuất trên Qdrant, reranking và sinh câu trả lời dạng streaming.

Kho tri thức của hệ thống được lưu trong Qdrant dưới dạng khoảng 270.000 vector 1024 chiều, được tạo từ corpus dinh dưỡng tiếng Việt. Khi có câu hỏi, LLM/RAG Service không đưa trực tiếp toàn bộ dữ liệu vào LLM mà tiến hành truy xuất ngữ nghĩa trên Qdrant để lấy các chunk liên quan. Sau đó, reranker đánh giá lại các chunk ứng viên và chọn những chunk phù hợp nhất làm ngữ cảnh đầu vào cho LLM. Quy trình này giúp câu trả lời dựa trên cơ sở tri thức đã xây dựng thay vì chỉ dựa vào kiến thức chung của mô hình ngôn ngữ.

Ở chiều đầu ra, LLM/RAG Service không chờ sinh xong toàn bộ câu trả lời. Các đoạn text được sinh dần dưới dạng streaming và chuyển về Gateway. Gateway gom văn bản thành các text chunk đủ điều kiện, sau đó gửi từng chunk sang TTS Service. TTS Service tổng hợp từng đoạn thành PCM 24 kHz và trả audio stream về Gateway để phát lại cho Web Client. Nhờ đó, quá trình sinh văn bản và tổng hợp giọng nói được chồng lấp thời gian, giúp giảm Time-to-First-Audio so với pipeline tuần tự.

Redis được sử dụng như session store để lưu lịch sử hội thoại ngắn hạn, trạng thái phiên và dữ liệu phục vụ tóm tắt hội thoại khi cần. Thành phần này giúp hệ thống duy trì ngữ cảnh giữa các lượt hỏi đáp mà không phải đưa toàn bộ lịch sử dài vào prompt. Ngoài ra, Gateway còn xử lý barge-in: khi VAD phát hiện người dùng bắt đầu nói trong lúc bot đang trả lời, Gateway hủy luồng TTS hiện tại, reset hàng đợi liên quan đến session cũ và chuyển pipeline sang câu hỏi mới.

Thiết kế pipeline mô-đun mang lại ba lợi ích chính. Thứ nhất, mỗi thành phần có thể được phát triển, kiểm thử và tối ưu độc lập. Thứ hai,

hệ thống có thể triển khai linh hoạt trên CPU hoặc GPU tùy yêu cầu hiệu năng của từng mô hình. Thứ ba, khi cần thay đổi một mô hình hoặc thuật toán, ví dụ thay backend LLM, thay mô hình ASR hoặc điều chỉnh chiến lược reranking, các thành phần còn lại không phải thay đổi lớn. Điểm quan trọng là hệ thống vẫn có một Gateway điều phối trung tâm và các worker phục vụ cùng một luồng nghiệp vụ, do đó thuật ngữ pipeline mô-đun phản ánh đúng kiến trúc hiện tại hơn so với việc mô tả như một hệ thống phân tán độc lập phức tạp.

Các thành phần chính của hệ thống được tổ chức như sau:

Bảng 3.1: Các thành phần chính trong hệ thống

Thành phần	Cổng	Vai trò
Gateway	8000	Điều phối phiên hội thoại, quản lý WebSocket, truyền audio, transcript, text chunk và audio chunk giữa client và các worker.
ASR	50051	Nhận audio PCM 16 kHz, thực hiện VAD và nhận dạng giọng nói tiếng Việt để tạo transcript.
LLM/RAG	50052	Thực hiện query expansion, HyDE, embedding, truy xuất Qdrant, reranking, xây dựng prompt và stream câu trả lời.
TTS	50053	Nhận text chunk, tổng hợp giọng nói bằng VieNeu-TTS và stream PCM 24 kHz về Gateway.
Qdrant	6333	Lưu trữ khoảng 270.000 vector embedding của kho tri thức dinh dưỡng và phục vụ truy vấn ngữ nghĩa.
Redis	6379	Lưu session memory, lịch sử hội thoại ngắn hạn và trạng thái phiên khi khả dụng.

Trong mã nguồn hiện tại, các thành phần Gateway, LLM/RAG, ASR, TTS và giao diện web được tách thành các module riêng. Docker Compose định nghĩa healthcheck cho từng worker, giúp Gateway chỉ khởi động khi các service phụ thuộc đã sẵn sàng. Cách tổ chức này cũng tạo điều kiện cho việc đo latency theo từng mô-đun, đánh giá nút thắt của pipeline và mở rộng từng worker theo nhu cầu thực tế.

3.1.1 Yêu cầu thiết kế của giải pháp

Từ bài toán đã phát biểu ở Chương 1, giải pháp cần thỏa mãn bốn nhóm yêu cầu chính. Thứ nhất là yêu cầu về tính thời gian thực: người dùng không nên phải chờ đến khi toàn bộ câu trả lời được sinh và tổng hợp xong mới nghe được phản hồi đầu tiên. Vì vậy, kiến trúc phải hỗ trợ streaming ở cả chiều văn bản và chiều âm thanh. Thứ hai là yêu cầu về tính có căn cứ: với chủ đề dinh dưỡng, câu trả lời cần dựa trên kho tri thức đã thu thập, hạn chế trả lời theo suy đoán chung của LLM. Thứ ba là yêu cầu về độ bền của hội thoại: hệ thống cần duy trì session, xử lý lượt hỏi liên tiếp, cho phép người dùng ngắt lời bot và tránh trộn dữ liệu giữa các phiên. Thứ tư là yêu cầu về khả năng đánh giá: mỗi bước trong pipeline cần có điểm đo để phân tích chất lượng và độ trễ.

Các yêu cầu trên dẫn tới lựa chọn kiến trúc pipeline mô-đun thay vì một khối xử lý duy nhất. Nếu toàn bộ ASR, RAG và TTS được đặt trong cùng một tiến trình, việc thay đổi mô hình, đo latency riêng từng thành phần và xử lý lỗi cục bộ sẽ khó hơn. Ngược lại, khi tách thành các worker riêng, Gateway có thể đóng vai trò điều phối luồng dữ liệu, còn từng worker tập trung vào một nhiệm vụ rõ ràng. Cách tách này không nhằm khẳng định hệ thống đã là một kiến trúc microservice hoàn chỉnh, mà nhằm tạo cấu trúc đủ rõ để phát triển, đánh giá và thay thế mô hình.

3.1.2 Luồng dữ liệu và trạng thái phiên

Trong một phiên hội thoại, dữ liệu đi qua nhiều dạng biểu diễn khác nhau: audio thô từ micro, transcript sau ASR, truy vấn đã chuẩn hóa, danh sách chunk truy xuất, câu trả lời dạng văn bản và audio tổng hợp. Nếu không quản lý trạng thái phiên rõ ràng, một lỗi thường gặp là audio hoặc text của lượt hỏi trước tiếp tục được phát trong khi người dùng đã bắt đầu lượt hỏi mới. Vì vậy, Gateway cần gắn mỗi luồng xử lý với một

session cụ thể và điều phối việc hủy luồng cũ khi có tín hiệu barge-in.

Về mặt dữ liệu, pipeline được tổ chức theo nguyên tắc mỗi mô-đun chỉ nhận loại dữ liệu cần thiết cho nhiệm vụ của mình. ASR chỉ xử lý audio và trả transcript; mô-đun xử lý hội thoại chỉ nhận câu hỏi văn bản, lịch sử ngắn hạn và trả text stream; TTS chỉ nhận văn bản đã làm sạch và trả audio. Việc phân tách này giúp giảm phụ thuộc chéo giữa các mô-đun. Ví dụ, TTS không cần biết tài liệu nào đã được truy xuất, còn ASR không cần biết prompt của LLM. Nhờ đó, khi thay đổi một thành phần, ảnh hưởng đến các thành phần còn lại được giới hạn.

3.2 Mô-đun xử lý tiếng nói

Mô-đun xử lý tiếng nói gồm hai hướng chuyển đổi: speech-to-text và text-to-speech. Ở chiều đầu vào, ASR nhận các chunk audio từ micro, chuẩn hóa về định dạng PCM 16 kHz mono và sinh transcript. Transcript này có thể chứa lỗi do nhiều môi trường, cách phát âm hoặc thuật ngữ chuyên ngành. Vì vậy, hệ thống áp dụng bước chuẩn hóa truy vấn để sửa một số lỗi thường gặp, ví dụ chuẩn hóa “omega ba” thành “omega 3” hoặc “can xi” thành “canxi”.

Ở chiều đầu ra, TTS nhận các đoạn văn bản ngắn, đã được làm sạch khỏi markdown hoặc ký hiệu không phù hợp với giọng đọc. Các đoạn văn bản được tổng hợp thành PCM 24 kHz và stream về client. Để tránh ngắt giọng thiếu tự nhiên, hệ thống chỉ cắt văn bản tại các vị trí tương đối an toàn như dấu kết thúc câu hoặc khi buffer đạt kích thước tối đa.

Luồng xử lý audio trong trình duyệt được tổ chức theo cơ chế streaming. Ở chiều gửi, client thu âm theo các frame ngắn, chuyển tín hiệu về định dạng PCM 16-bit phù hợp với ASR và gửi liên tục qua WebSocket. Ở chiều nhận, client nhận các chunk PCM 24 kHz từ Gateway, đưa vào bộ đệm phát âm thanh và lập lịch phát nối tiếp để các

đoạn audio rời rạc được nghe gần như liên tục. Cách tổ chức này giúp giảm khoảng lặng giữa các đoạn, đồng thời phù hợp với việc TTS trả audio theo từng phần thay vì một file hoàn chỉnh.

TTS còn hỗ trợ hủy tổng hợp theo session thông qua một API điều khiển riêng. Chức năng này phục vụ barge-in: khi người dùng bắt đầu nói trong lúc bot đang trả lời, Gateway có thể hủy audio đang phát, reset queue và chuyển sang xử lý câu hỏi mới. Đây là yêu cầu quan trọng đối với trải nghiệm hội thoại tự nhiên.

3.2.1 Thiết kế chiều nhận dạng giọng nói

Ở chiều đầu vào, mục tiêu của ASR không chỉ là tạo transcript chính xác, mà còn phải trả kết quả đủ nhanh để không làm chậm toàn bộ pipeline. Do đó, audio từ trình duyệt được chia thành các frame ngắn và gửi liên tục thay vì đợi người dùng nói xong rồi mới tải một file âm thanh hoàn chỉnh. Cách làm này cho phép hệ thống phát hiện đoạn có tiếng nói, xác định điểm kết thúc lượt nói và bắt đầu xử lý hội thoại ngay khi transcript cuối cùng sẵn sàng.

Một vấn đề quan trọng của ASR tiếng Việt trong chủ đề dinh dưỡng là lỗi nhận dạng không phân bố ngẫu nhiên. Các lỗi thường tập trung vào từ mượn, ký hiệu số, đơn vị đo, tên chất dinh dưỡng hoặc cách nói đời thường. Ví dụ, người dùng có thể nói “omega ba”, “can xi”, “vitamin đề”, trong khi tài liệu viết là “omega-3”, “canxi”, “vitamin D”. Vì vậy, phần xử lý phía sau ASR không xem transcript là hoàn toàn sạch, mà cần có bước chuẩn hóa nhẹ trước khi truy xuất tri thức.

3.2.2 Thiết kế chiều tổng hợp giọng nói

Ở chiều đầu ra, TTS được đặt sau bước sinh câu trả lời và là thành phần trực tiếp quyết định cảm nhận của người dùng. Nếu câu trả lời được

gửi sang TTS dưới dạng một đoạn dài, âm thanh đầu tiên sẽ xuất hiện muộn. Nếu chia câu trả lời thành quá nhiều đoạn nhỏ, âm thanh có thể bị rời rạc. Vì vậy, thiết kế TTS trong đồ án tập trung vào việc nhận các đoạn văn bản vừa đủ ngắn để phản hồi nhanh, nhưng vẫn đủ dài để giữ ngữ điệu tự nhiên.

Trước khi gửi văn bản sang TTS, Gateway thực hiện làm sạch đầu ra của LLM. Các ký hiệu markdown, bullet, tiêu đề, URL hoặc định dạng không phù hợp với giọng đọc được loại bỏ hoặc chuyển sang dạng dễ đọc hơn. Đây là bước cần thiết vì LLM thường sinh văn bản theo phong cách màn hình, trong khi TTS cần đầu vào theo phong cách lời nói. Nếu bỏ qua bước này, bot có thể đọc các ký hiệu thừa, làm giảm tính tự nhiên của hội thoại.

3.3 Mô-đun xử lý hội thoại

Mô-đun xử lý hội thoại chịu trách nhiệm biến transcript của người dùng thành câu trả lời có căn cứ. Trong hệ thống này, LLM không trả lời trực tiếp từ câu hỏi thô, mà được đặt trong một RAG pipeline gồm bước tiền xử lý truy vấn và ba bước chính tương ứng với Retrieval, Augmentation và Generation.

- 1) **Tiền xử lý truy vấn:** transcript được chuẩn hóa bằng query expansion dựa trên từ điển thuật ngữ dinh dưỡng và các biến thể diễn đạt thường gặp. Ví dụ, các cách hỏi phổ thông như “tiểu đường”, “bà bầu”, “omega 3” hoặc “béo phì” được mở rộng sang các thuật ngữ liên quan như “đái tháo đường”, “phụ nữ mang thai”, “axit béo omega-3”, “BMI cao” hoặc “thừa cân”. Bước này giúp giảm khoảng cách giữa cách người dùng nói tự nhiên và cách thuật ngữ xuất hiện trong tài liệu chuyên ngành.
- 2) **Retrieval:** câu hỏi sau chuẩn hóa được mã hóa bằng mô hình

embedding tiếng Việt AITeamVN/Vietnamese_Embedding để truy xuất ngữ nghĩa trên Qdrant. Ở bước đầu, hệ thống lấy 15 chunk ứng viên từ kho tri thức. Sau đó, mô hình reranker tiếng Việt đánh giá lại từng cặp câu hỏi–chunk và chọn 5 chunk tốt nhất. Đây là quy trình truy xuất hai bước, trong đó mô hình embedding ưu tiên tốc độ và độ phủ, còn reranker giúp tăng độ chính xác của ngữ cảnh được chọn.

- 3) **Augmentation:** 5 chunk tốt nhất sau reranking được ghép vào prompt dưới dạng context cho LLM. Ngoài context truy xuất, prompt còn có câu hỏi hiện tại đã chuẩn hóa, hướng dẫn trả lời theo chủ đề dinh dưỡng và tóm tắt hội thoại gần nhất nếu có. Như vậy, LLM có context rõ ràng để sinh câu trả lời dựa trên tài liệu thay vì chỉ dựa vào kiến thức chung.
- 4) **Generation:** LLM sinh câu trả lời dựa trên prompt đã được bổ sung context. Câu trả lời cần đúng trọng tâm câu hỏi, bám sát ngữ cảnh truy xuất, ngắn gọn, dễ nghe khi chuyển sang giọng nói và không đọc trực tiếp URL hoặc metadata nguồn trong câu trả lời thoại.

Ngoài query expansion dựa trên từ điển, hệ thống có thể sử dụng HyDE để tạo một biểu diễn truy vấn giàu ngữ nghĩa hơn trước khi truy xuất. Mục tiêu của bước này là tăng khả năng tìm được các chunk liên quan trong trường hợp câu hỏi người dùng ngắn, diễn đạt đời thường hoặc thiếu thuật ngữ chuyên môn.

Luồng sinh câu trả lời được tổ chức theo streaming. LLM vừa sinh từng phần văn bản vừa trả kết quả về Gateway, thay vì đợi toàn bộ câu trả lời hoàn tất. Trên phần đầu tiên, hệ thống ghi nhận các thông tin timing như thời gian tiền xử lý truy vấn, thời gian truy xuất–reranking, thời gian nhận phần trả lời đầu tiên từ LLM và thời gian tổng thể đến phần đầu tiên. Những thông tin này được dùng để phân tích nút thắt latency của mô-đun xử lý hội thoại.

Về bộ nhớ hội thoại, Gateway ưu tiên Redis nếu có kết nối. Mỗi session lưu các lượt gần nhất và một bản tóm tắt hội thoại cũ. Khi Redis không khả dụng, Gateway dùng fallback memory trong RAM. Điều này giúp hệ thống vẫn chạy trong môi trường phát triển cục bộ, đồng thời có khả năng mở rộng tốt hơn khi triển khai với Redis.

3.3.1 Tổ chức ba bước Retrieval, Augmentation và Generation

Ba bước R, A và G có vai trò khác nhau và cần được tách rõ trong thiết kế. Retrieval quyết định hệ thống lấy được bằng chứng nào; Augmentation quyết định bằng chứng đó được đưa vào đầu vào của LLM theo cấu trúc nào; Generation quyết định cách LLM sử dụng bằng chứng để trả lời. Nếu Retrieval sai, LLM có thể không có đủ tài liệu để trả lời. Nếu Augmentation kém, tài liệu đúng vẫn có thể bị mô hình bỏ qua. Nếu Generation không được ràng buộc tốt, mô hình có thể diễn giải vượt quá ngữ cảnh.

Bảng 3.2: Vai trò của ba bước trong RAG pipeline

Bước	Đầu vào chính	Kết quả mong muốn
Retrieval	Câu hỏi đã chuẩn hóa và embedding truy vấn	Danh sách chunk có liên quan cao tới câu hỏi
Augmentation	Chunk đã reranking, câu hỏi hiện tại và lịch sử ngắn hạn	Prompt có ngữ cảnh rõ ràng, vừa đủ dài và đúng phạm vi
Generation	Prompt đã bổ sung ngữ cảnh	Câu trả lời ngắn gọn, có căn cứ, phù hợp để đọc thành giọng nói

Trong đồ án, bước Retrieval được thiết kế theo hướng ưu tiên độ phủ ở pha đầu và độ chính xác ở pha sau. Embedding search lấy tập ứng viên đủ rộng để tránh bỏ sót tài liệu liên quan. Reranking sau đó giảm tập này xuống các chunk có mức phù hợp cao hơn. Bước Augmentation kiểm soát cách đưa context vào prompt: context được gom theo thứ tự ưu tiên sau reranking, đồng thời tránh đưa quá nhiều văn bản làm prompt dài và tăng độ trễ LLM. Bước Generation yêu cầu LLM trả lời theo phong cách hội thoại, không liệt kê rườm rà và không đọc metadata nguồn trong câu

trả lời thoại.

3.3.2 Kiểm soát phạm vi và chất lượng câu trả lời

Do chủ đề dinh dưỡng nằm gần lĩnh vực y tế, hệ thống không nên trả lời mọi câu hỏi bằng cùng một chiến lược. Các câu hỏi về chế độ ăn thông thường có thể được trả lời dựa trên RAG. Các câu hỏi có dấu hiệu khẩn cấp, yêu cầu chẩn đoán hoặc yêu cầu thay đổi thuốc cần được chuyển sang phản hồi an toàn. Các câu hỏi ngoài chủ đề dinh dưỡng cần được từ chối hoặc yêu cầu người dùng đặt lại câu hỏi phù hợp hơn.

Kiểm soát chất lượng câu trả lời được thực hiện bằng cách kết hợp ba lớp. Lớp thứ nhất là guardrail đầu vào để phát hiện câu hỏi rủi ro trước khi truy xuất. Lớp thứ hai là context từ RAG, giúp LLM có bằng chứng cụ thể. Lớp thứ ba là ràng buộc prompt, yêu cầu mô hình trả lời ngắn, đúng trọng tâm, không khẳng định quá mức và khuyến nghị gặp chuyên gia khi câu hỏi vượt phạm vi. Cách tổ chức này thực dụng hơn việc thêm nhiều chain LLM phụ, vì mỗi lượt gọi bổ sung đều làm tăng latency.

3.4 Tối ưu pipeline Voice RAG dưới ràng buộc độ trễ

RAG trong chatbot văn bản thường tập trung vào việc truy xuất đủ tài liệu và sinh câu trả lời chính xác. Với voicebot thời gian thực, bài toán có thêm một ràng buộc quan trọng: mọi bước xử lý đều ảnh hưởng trực tiếp đến thời gian người dùng phải chờ trước khi nghe được phản hồi. Vì vậy, đề án không chỉ xem RAG là một bước truy xuất độc lập, mà đặt RAG trong toàn bộ pipeline giọng nói, nơi transcript từ ASR, thời gian truy xuất, thời gian sinh token đầu tiên của LLM và thời gian tổng hợp âm thanh đều cùng quyết định trải nghiệm hội thoại.

Luồng xử lý được đề xuất có thể mô tả ở mức khái quát như sau:

User speech → *ASR* → *Input guardrail* → *Query normalization*
→ *Query expansion* → *Retrieval* → *Reranking* → *Evidence control*
→ *Grounded LLM generation*
→ *TTS streaming* → *Voice answer*

Trong đó, *input guardrail* là bước kiểm tra đầu vào trước khi truy xuất. Các câu hỏi có dấu hiệu khẩn cấp, yêu cầu chẩn đoán, kê đơn hoặc vượt phạm vi tư vấn dinh dưỡng được chuyển sang phản hồi an toàn thay vì để LLM tự sinh câu trả lời chi tiết. Cách làm này không nhằm thay thế bác sĩ, mà nhằm giới hạn phạm vi của hệ thống và giảm rủi ro trả lời quá mức trong một chủ đề liên quan đến sức khỏe.

Sau *guardrail*, *query normalization* chuẩn hóa câu hỏi sau nhận dạng giọng nói. Đây là bước xử lý nhẹ, không yêu cầu thêm lượt gọi LLM. Hệ thống loại bỏ một số từ đệm trong hội thoại, chuẩn hóa các cách nói phổ thông sang thuật ngữ dinh dưỡng và sửa một số lỗi nhận dạng thường gặp như “omega ba” thành “omega 3” hoặc “can xi” thành “canxi”. Với các thuật ngữ có nhiều cách diễn đạt, *query expansion* bổ sung thêm biến thể truy vấn để tăng khả năng khớp với tài liệu trong kho tri thức.

Ở bước truy xuất, hệ thống lấy một số lượng chunk ứng viên từ Qdrant bằng *embedding search*, sau đó dùng *reranker* để chọn các chunk phù hợp nhất đưa vào đầu vào của LLM. Cách làm hai bước này giúp cân bằng giữa độ phủ của truy xuất ban đầu và độ chính xác của ngữ cảnh cuối cùng. Nếu tài liệu truy xuất quá ít hoặc nội dung không đủ căn cứ, bước kiểm soát bằng chứng giúp hệ thống tránh trả lời chắc chắn trên ngữ cảnh yếu.

Bảng dưới đây tóm tắt vai trò của từng cơ chế tối ưu trong pipeline Voice RAG.

Bảng 3.3: Các cơ chế tối ưu trong pipeline Voice RAG

Cơ chế	Mục tiêu	Tác động đến hệ thống
Input guardrail	Kiểm tra câu hỏi trước khi truy xuất	Giới hạn phạm vi tư vấn và giảm rủi ro trả lời quá mức
Query normalization	Giảm nhiễu từ transcript giọng nói	Cải thiện truy xuất với câu hỏi tiếng Việt ngắn hoặc có lỗi nhận dạng
Retrieval và reranking	Chọn các chunk phù hợp nhất từ kho tri thức	Cân bằng giữa độ phủ ngữ cảnh và độ dài đầu vào của LLM
Evidence control	Kiểm tra ngữ cảnh có đủ để trả lời không	Giảm rủi ro LLM suy diễn khi tài liệu truy xuất yếu

Điểm quan trọng của hướng tối ưu này là hạn chế thêm các chain LLM phụ. Các bước guardrail, chuẩn hóa truy vấn, query expansion, retrieval, reranking và kiểm soát bằng chứng được tổ chức trước khi sinh câu trả lời, trong khi streaming LLM–TTS được dùng để giảm thời gian chờ âm thanh đầu tiên. Nhờ đó, hệ thống cải thiện khả năng vận hành thực tế mà không đánh đổi quá nhiều độ trễ.

3.4.1 Phân tích trade-off chất lượng và độ trễ

Trong hệ thống RAG, tăng số lượng tài liệu truy xuất thường có thể cải thiện khả năng bao phủ thông tin, nhưng đồng thời làm tăng chi phí reranking và làm dài đầu vào của LLM. Với chatbot văn bản, độ trễ tăng thêm có thể vẫn chấp nhận được. Với voicebot, độ trễ này ảnh hưởng trực tiếp tới thời gian người dùng chờ trong im lặng. Vì vậy, đề án chọn cách truy xuất hai bước: lấy một tập ứng viên đủ rộng để không bỏ sót tài liệu quan trọng, sau đó chỉ đưa một số chunk tốt nhất vào LLM.

Tương tự, ở bước sinh câu trả lời, việc yêu cầu LLM trả lời quá đầy đủ có thể làm tăng chất lượng thông tin nhưng kéo dài thời gian sinh và thời gian TTS. Vì vậy, câu trả lời được định hướng theo phong cách ngắn gọn, ưu tiên thông tin cần thiết, tránh liệt kê dài và tránh đọc các chi tiết không phù hợp với kênh thoại. Đây là khác biệt quan trọng giữa RAG cho giao diện đọc và RAG cho giao diện nghe.

3.4.2 Kiểm soát bằng chứng ở mức thực dụng

Kiểm soát bằng chứng trong đồ án được hiểu là kiểm tra xem hệ thống có đủ ngữ cảnh hợp lý để sinh câu trả lời hay không, thay vì khẳng định có một bộ kiểm chứng y khoa hoàn chỉnh. Ở mức hiện tại, cơ chế này dựa trên kết quả truy xuất, điểm phù hợp sau reranking và ràng buộc prompt. Nếu ngữ cảnh yếu hoặc câu hỏi vượt quá phạm vi corpus, hệ thống nên trả lời theo hướng thận trọng, nói rõ giới hạn thông tin và khuyến nghị hỏi chuyên gia.

Cách tiếp cận này phù hợp với phạm vi đồ án vì không làm tăng đáng kể latency. Một evidence checker đầy đủ có thể cần thêm một lượt gọi LLM hoặc một mô hình phân loại riêng, nhưng điều đó sẽ làm pipeline chậm hơn. Do đó, phiên bản hiện tại ưu tiên kiểm soát bằng chứng bằng các tín hiệu sẵn có trong pipeline, còn các bộ kiểm chứng nâng cao được xem là hướng phát triển sau.

3.5 Cơ chế streaming song song

Để tối ưu độ trễ, hệ thống không xử lý câu trả lời theo kiểu tuần tự mà sử dụng cơ chế streaming song song giữa LLM và TTS. Về nguyên tắc, LLM không cần sinh xong toàn bộ câu trả lời mới chuyển sang TTS. Thay vào đó, khi LLM sinh được một phần văn bản đủ điều kiện, Gateway có thể đưa phần văn bản này vào hàng đợi để TTS bắt đầu tổng hợp âm thanh. Nhờ vậy, thời gian sinh văn bản và thời gian tổng hợp giọng nói được chồng lấp, làm giảm thời gian người dùng phải chờ trước khi nghe được âm thanh đầu tiên.

Cơ chế này được tổ chức theo mô hình producer-consumer. LLM đóng vai trò producer, liên tục sinh các đoạn văn bản mới. TTS đóng vai trò consumer, lấy từng đoạn văn bản từ hàng đợi bất đồng bộ và tổng hợp audio ngay khi có thể. Gateway nằm giữa hai thành phần này, vừa nhận

text stream từ LLM, vừa quản lý hàng đợi text cho TTS và hàng đợi event trả về client. Trong hiện thực, hàng đợi bất đồng bộ có thể được triển khai bằng cấu trúc như `asyncio.Queue`, nhưng về mặt kiến trúc, điểm quan trọng là producer và consumer có thể chạy song song thay vì chờ nhau hoàn toàn.

Kỹ thuật word-safe chunking được sử dụng để quyết định khi nào một phần văn bản đủ điều kiện gửi sang TTS. Nếu chunk quá ngắn, TTS có thể đọc bị ngắt nhịp hoặc tạo nhiều đoạn audio nhỏ làm tăng overhead. Nếu chunk quá dài, hệ thống phải chờ lâu hơn trước khi bắt đầu tổng hợp âm thanh, làm tăng Time-to-First-Audio. Vì vậy, hệ thống đặt ngưỡng ký tự tối thiểu, đồng thời ưu tiên cắt tại các vị trí tự nhiên như dấu chấm, dấu hỏi, dấu chấm than hoặc dấu phẩy. Trong cấu hình thực nghiệm, ngưỡng tối thiểu 40 ký tự được sử dụng như một điểm cân bằng giữa phản hồi nhanh và giữ ngữ điệu tương đối tự nhiên.

Thuật toán streaming trong Gateway có thể mô tả như sau:

- 1) Gateway khởi tạo hàng đợi text cho TTS và hàng đợi event để trả kết quả về client.
- 2) LLM sinh câu trả lời theo luồng, Gateway gửi phần text mới về client và đồng thời cộng dồn vào buffer.
- 3) Khi buffer đạt ngưỡng tối thiểu và gặp điểm ngắt tự nhiên, hoặc khi buffer vượt ngưỡng tối đa, Gateway cắt một text chunk an toàn theo ranh giới từ.
- 4) Text chunk được đưa vào hàng đợi TTS để tổng hợp thành audio mà không cần chờ toàn bộ câu trả lời hoàn tất.
- 5) Audio chunk từ TTS được đưa vào hàng đợi event và stream về trình duyệt theo WebSocket.

Chiến thuật này làm thay đổi bản chất latency. Với pipeline tuần tự,

thời gian nghe được âm thanh đầu tiên xấp xỉ tổng thời gian sinh toàn bộ câu trả lời cộng với thời gian TTS. Với pipeline streaming, thời gian này chỉ phụ thuộc vào thời gian sinh chunk đầu tiên đủ tốt và thời gian TTS tạo frame đầu tiên cho chunk đó. Vì vậy, tối ưu chunking là một trong các đóng góp quan trọng của đề án.

3.5.1 Mô hình thời gian của pipeline

Có thể mô tả trực quan sự khác biệt giữa pipeline tuần tự và pipeline streaming bằng mô hình thời gian. Với pipeline tuần tự, thời gian đến âm thanh đầu tiên có thể xấp xỉ:

$$T_{\text{seq}} = T_{\text{ASR}} + T_{\text{RAG}} + T_{\text{LLM-full}} + T_{\text{TTS-first}}$$

Trong đó $T_{\text{LLM-full}}$ là thời gian LLM sinh xong toàn bộ câu trả lời. Với pipeline streaming, hệ thống không cần chờ toàn bộ câu trả lời, nên thời gian đến âm thanh đầu tiên có thể xấp xỉ:

$$T_{\text{stream}} = T_{\text{ASR}} + T_{\text{RAG}} + T_{\text{LLM-first-chunk}} + T_{\text{TTS-first}}$$

Vì $T_{\text{LLM-first-chunk}}$ thường nhỏ hơn đáng kể so với $T_{\text{LLM-full}}$, streaming có thể giảm mạnh Time-to-First-Audio ngay cả khi tổng lượng tính toán không đổi. Công thức này cũng cho thấy nếu ASR hoặc RAG quá chậm, streaming LLM–TTS chỉ giải quyết được một phần vấn đề. Do đó, đánh giá Chương 4 cần tách latency theo từng thành phần.

3.5.2 Đảm bảo tính liên mạch của audio

Khi TTS trả về nhiều audio chunk rời rạc, client cần phát chúng theo thứ tự và tránh tạo khoảng trống giữa các chunk. Vì vậy, trình duyệt không phát audio ngay lập tức một cách độc lập cho từng chunk, mà

đưa vào hàng đợi phát và lập lịch nối tiếp. Nếu chunk mới đến trước khi chunk cũ phát xong, nó được xếp sau. Nếu chunk đến muộn, người dùng có thể nghe thấy khoảng lặng. Do đó, trải nghiệm cuối cùng phụ thuộc đồng thời vào tốc độ sinh text, tốc độ TTS và cách client lập lịch phát audio.

Thiết kế này cũng giúp phân biệt rõ hai loại độ trễ. Time-to-First-Audio đo thời điểm âm thanh đầu tiên xuất hiện, còn độ mượt khi nghe phụ thuộc vào việc các audio chunk tiếp theo có được tạo đủ nhanh để nối tiếp nhau hay không. Một cấu hình chunk quá nhỏ có thể cải thiện Time-to-First-Audio nhưng làm tăng số lần gọi TTS và tăng nguy cơ audio không liên mạch. Vì vậy, lựa chọn kích thước chunk cần được đánh giá bằng cả số liệu latency và nghe thử.

3.6 Các tính năng để hệ thống có thể triển khai thực tế

Để dự án không chỉ dừng ở mức ghép nối mô hình, hệ thống cần có các tính năng bảo vệ trải nghiệm người dùng, giảm rủi ro tư vấn sai và hỗ trợ vận hành. Trong hiện thực hiện tại, các nhóm tính năng đã có nền tảng kỹ thuật rõ ràng gồm guardrail an toàn, bộ nhớ hội thoại, barge-in và khả năng quan sát latency.

Tính năng thứ nhất là guardrail an toàn y tế ở khối LLM. Trước khi truy xuất và sinh câu trả lời, câu hỏi của người dùng được kiểm tra bằng một lớp luật nhẹ để phát hiện ba nhóm tình huống: dấu hiệu khẩn cấp như khó thở, đau ngực, ngất hoặc co giật; yêu cầu vượt thẩm quyền như chẩn đoán, kê đơn, đổi liều thuốc; và câu hỏi ngoài phạm vi dinh dưỡng. Khi guardrail được kích hoạt, hệ thống không truy xuất RAG và không để LLM tự sinh tư vấn chi tiết, mà trả về thông điệp an toàn, khuyến nghị người dùng liên hệ bác sĩ hoặc đặt lại câu hỏi trong phạm vi dinh dưỡng. Đây là yêu cầu quan trọng đối với lĩnh vực sức khỏe, vì lỗi nghiêm trọng

không chỉ là trả lời sai mà còn là tạo cảm giác hệ thống có thể thay thế bác sĩ.

Tính năng thứ hai là bộ nhớ hội thoại theo phiên. Một người dùng thực tế hiếm khi hỏi một câu độc lập; họ có thể hỏi tiếp “vậy tôi nên ăn bao nhiêu”, “còn trẻ em thì sao” hoặc “tôi bị tiểu đường thì có khác không”. Vì vậy, Gateway lưu các lượt hội thoại gần nhất và tóm tắt các lượt cũ hơn. Khi Redis khả dụng, lịch sử được lưu theo session với thời gian sống giới hạn; khi chạy cục bộ, hệ thống dùng fallback memory trong RAM. Cách thiết kế này giúp hội thoại có ngữ cảnh nhưng vẫn kiểm soát được độ dài prompt.

Tính năng thứ ba là barge-in, tức khả năng người dùng cắt lời bot. Trong hội thoại thật, người dùng có thể nhận ra bot hiểu sai câu hỏi hoặc muốn hỏi thêm trước khi bot đọc xong. Gateway kết hợp VAD streaming và tín hiệu hủy TTS để dừng audio hiện tại khi phát hiện người dùng bắt đầu nói. Nếu không có barge-in, hệ thống sẽ giống một trình đọc văn bản hơn là một trợ lý hội thoại.

Tính năng thứ tư là truy vết nguồn và quan sát hệ thống. RAG lưu metadata như tiêu đề, nguồn và URL cho mỗi chunk. Dù câu trả lời thoại không đọc URL để tránh gây khó nghe, metadata vẫn cần được giữ lại cho giao diện debug, đánh giá chất lượng và kiểm tra nguồn khi có phản hồi xấu. Bên cạnh đó, khối LLM trả timing cho các giai đoạn query expansion, RAG, token đầu tiên và tổng thời gian xử lý. Các số liệu này là cơ sở để xác định nút thắt khi triển khai trên máy thật.

Để hệ thống tiến gần hơn tới sản phẩm hoàn chỉnh, các tính năng tiếp theo nên được ưu tiên theo thứ tự thực dụng. Trước hết là giao diện hiển thị nguồn tham khảo ở dạng thẻ sau câu trả lời, chỉ dùng cho người đọc màn hình chứ không đọc qua TTS. Tiếp theo là cơ chế người dùng phản hồi theo mức hữu ích hoặc không hữu ích để thu thập lỗi RAG và lỗi

ASR. Sau đó là trang quản trị corpus cho phép thêm tài liệu mới, rebuild embedding và kiểm tra câu hỏi nào không tìm được ngữ cảnh phù hợp. Cuối cùng là logging theo session với các trường transcript, tài liệu truy xuất, câu trả lời, mã guardrail và latency từng bước. Những tính năng này không làm thay đổi thuật toán cốt lõi, nhưng quyết định việc hệ thống có thể được vận hành, kiểm tra và cải thiện liên tục trong thực tế hay không.

3.6.1 Nguyên tắc xử lý lỗi

Một hệ thống voicebot thực tế cần xử lý được các lỗi cục bộ mà không làm hỏng toàn bộ phiên hội thoại. Nếu ASR không nhận ra lời nói, hệ thống nên yêu cầu người dùng nói lại thay vì chuyển một transcript rỗng sang LLM. Nếu truy xuất không tìm được ngữ cảnh đủ tốt, hệ thống nên trả lời thận trọng và nói rõ giới hạn. Nếu TTS lỗi, giao diện vẫn nên hiển thị câu trả lời văn bản để người dùng không mất hoàn toàn kết quả. Các nhánh lỗi này giúp hệ thống có hành vi dự đoán được, thay vì để lỗi kỹ thuật biến thành câu trả lời sai.

3.6.2 Các giới hạn thiết kế

Giải pháp đề xuất vẫn có một số giới hạn cần nêu rõ. Thứ nhất, chất lượng cuối cùng phụ thuộc mạnh vào kho tri thức; hệ thống không thể trả lời tốt các câu hỏi mà corpus không bao phủ. Thứ hai, pipeline nhiều mô-đun tạo thêm chi phí truyền dữ liệu và chi phí điều phối so với một mô hình end-to-end. Thứ ba, các cơ chế an toàn ở hiện tại chủ yếu dựa trên luật nhẹ, prompt và context truy xuất, chưa phải một hệ thống kiểm định y khoa độc lập. Việc nêu rõ các giới hạn này giúp Chương 4 có cơ sở đánh giá thực nghiệm và giúp Chương 5 xác định hướng phát triển phù hợp.

CHƯƠNG 4. PHÁT TRIỂN HỆ THỐNG, THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Xây dựng kho tri thức

Kho tri thức dinh dưỡng được xây dựng từ các nguồn thông tin uy tín, sau đó trải qua các bước thu thập, làm sạch, chuẩn hóa và phân đoạn văn bản. Mục tiêu của quá trình này là tạo ra các chunk vừa đủ nhỏ để truy xuất chính xác, nhưng vẫn đủ ngữ cảnh để LLM sinh câu trả lời có ý nghĩa. Đây là bước nền tảng của hệ thống vì chất lượng RAG phụ thuộc trực tiếp vào độ tin cậy của nguồn dữ liệu và chất lượng phân đoạn văn bản.

Quy trình xử lý dữ liệu gồm các bước: loại bỏ nội dung nhiễu, chuẩn hóa unicode và khoảng trắng, tách câu hoặc đoạn theo ngữ nghĩa, tạo embedding cho từng chunk và lưu vào Qdrant. Với các tài liệu dài, chiến lược chunking cần tránh cắt giữa các ý quan trọng. Kích thước chunk là một tham số quan trọng vì ảnh hưởng đồng thời đến chất lượng truy xuất, chi phí embedding và độ chính xác của câu trả lời.

Corpus hiện tại gồm 1060 bài viết dinh dưỡng từ ba nguồn công khai có tính chuyên môn: Báo Sức khỏe và Đời sống, trang web của Viện Dinh dưỡng Quốc gia và Bệnh viện Đa khoa Quốc tế Thu Cúc. Mỗi bản ghi trong kho dữ liệu có các trường: nguồn, URL, tiêu đề, nội dung, chuyên mục và thời điểm thu thập. Nội dung được dùng để tạo kho tri thức cho các chủ đề như tăng chiều cao, chế độ ăn cho bệnh nền, vitamin, khoáng chất, dinh dưỡng cho trẻ em và phụ nữ mang thai. Sau khi thu thập, dữ liệu được làm sạch để loại bỏ nhiễu HTML, chuẩn hóa xuống dòng và ghép các đoạn rời rạc thành văn bản liền mạch.

Trong quá trình embedding, mỗi chunk được lưu cùng metadata như tiêu đề, nguồn và URL. Metadata không được đọc trực tiếp trong câu

trả lời thoại, nhưng vẫn có ích cho debug, đánh giá và truy vết nguồn dữ liệu. Khi triển khai bằng Docker, Qdrant có volume lưu trữ riêng và có cơ chế khôi phục snapshot. Điều này giúp hệ thống có thể tái lập collection mà không cần chạy lại toàn bộ pipeline thu thập, làm sạch và embedding dữ liệu.

4.2 Phát triển hệ thống

4.2.1 Thiết kế hệ thống

Thiết kế hệ thống trong giai đoạn phát triển bám theo giải pháp đã trình bày ở Chương 3. Hệ thống được tổ chức thành các khối độc lập gồm Web Client, Gateway, ASR, LLM/RAG, TTS, Qdrant và Redis. Gateway là điểm điều phối trung tâm, chịu trách nhiệm nhận audio từ client, gửi audio sang ASR, chuyển transcript sang LLM/RAG, nhận text stream từ LLM/RAG, chia văn bản thành các chunk phù hợp và gửi sang TTS để tổng hợp âm thanh.

Về công nghệ, Gateway và các worker phía sau được xây dựng bằng FastAPI để thuận tiện triển khai các luồng HTTP/WebSocket và streaming. Web Client được xây dựng bằng React để phục vụ thu âm, hiển thị trạng thái hội thoại và phát audio trả về. Qdrant được dùng làm vector database cho kho tri thức dinh dưỡng, Redis được dùng để lưu trạng thái phiên và lịch sử hội thoại ngắn hạn. Các mô hình AI được tách thành các worker riêng để thuận tiện kiểm thử, thay thế mô hình và đo latency theo từng thành phần.

Thiết kế này cho phép hệ thống chạy cục bộ trong quá trình phát triển, đồng thời có thể đóng gói bằng Docker để tái lập môi trường triển khai. Với môi trường có GPU, các thành phần nặng như ASR, embedding, reranking hoặc TTS có thể được tăng tốc bằng CUDA. Với môi trường CPU, hệ thống vẫn có thể chạy nhưng cần điều chỉnh mô hình, số luồng

xử lý và mức đồng thời phù hợp.

Luồng xử lý voice-to-voice trong giai đoạn phát triển được tổ chức theo thứ tự sau:

- 1) Web Client thu audio từ người dùng, chuẩn hóa định dạng và gửi liên tục về Gateway.
- 2) Gateway chuyển audio sang ASR để tạo transcript tiếng Việt cho từng lượt nói.
- 3) Transcript được gửi sang LLM/RAG để chuẩn hóa truy vấn, truy xuất tài liệu, reranking và sinh câu trả lời dạng streaming.
- 4) Gateway gom các đoạn văn bản đủ điều kiện và chuyển từng đoạn sang TTS để tổng hợp âm thanh.
- 5) Audio từ TTS được stream ngược về Gateway và phát trên trình duyệt theo đúng session của người dùng.

Các nhóm giao tiếp quan trọng trong hệ thống gồm:

- Gateway cung cấp kênh hội thoại thời gian thực, kiểm tra trạng thái hệ thống và điều phối session.
- ASR hỗ trợ nhận dạng theo audio hoàn chỉnh và nhận dạng theo luồng có VAD.
- LLM/RAG hỗ trợ query expansion, HyDE, truy xuất hai bước, sinh câu trả lời đầy đủ, sinh câu trả lời dạng streaming và nén lịch sử hội thoại.
- TTS hỗ trợ tổng hợp audio đầy đủ, tổng hợp PCM dạng streaming và hủy tổng hợp để phục vụ barge-in.

Việc tách chế độ batch và streaming giúp quá trình phát triển thuận tiện hơn. Chế độ batch phù hợp cho kiểm thử nhanh hoặc xử lý audio

hoàn chỉnh; chế độ streaming phù hợp cho hội thoại trực tiếp. Trong đánh giá latency, các luồng streaming là trọng tâm vì chúng phản ánh trải nghiệm người dùng thực tế.

4.2.2 Xây dựng hệ thống

Quá trình xây dựng hệ thống được thực hiện theo hướng phát triển và kiểm thử từng mô-đun trước khi tích hợp end-to-end. ASR được kiểm tra với audio mẫu và audio thu trực tiếp để đánh giá khả năng tạo transcript tiếng Việt. LLM/RAG được kiểm tra bằng các câu hỏi dinh dưỡng phổ biến nhằm đánh giá query expansion, HyDE, truy xuất tài liệu, reranking và khả năng sinh câu trả lời dựa trên context. TTS được kiểm tra bằng các đoạn văn bản có độ dài khác nhau để đánh giá thời gian tạo audio đầu tiên và độ tự nhiên khi đọc.

Sau khi từng thành phần hoạt động ổn định, hệ thống được tích hợp thành pipeline hoàn chỉnh từ Web Client đến Gateway, ASR, LLM/RAG, TTS và trả audio về trình duyệt. Các kịch bản kiểm thử đơn vị tập trung vào hàm xử lý văn bản, guardrail, query expansion, lựa chọn context và các logic streaming. Các kịch bản kiểm thử tích hợp tập trung vào luồng hỏi đáp hoàn chỉnh, khả năng phát audio liên tục, barge-in và việc giữ session memory giữa các lượt hội thoại.

Hệ thống hỗ trợ chạy cục bộ và triển khai bằng Docker. Docker Compose định nghĩa các thành phần Gateway, ASR, LLM/RAG, TTS, Qdrant, Redis và Web. Gateway phụ thuộc vào ASR, LLM/RAG, TTS và Redis. Worker LLM/RAG phụ thuộc vào Qdrant và cơ chế khôi phục dữ liệu vector. Mỗi worker AI có healthcheck riêng để phát hiện lỗi khởi động. Bản cấu hình GPU override cho phép tăng tốc phần cứng bằng CUDA khi máy chủ có GPU phù hợp.

Kết quả của giai đoạn xây dựng là hệ thống có thể thực hiện một lượt

kiểm thử voice-to-voice hoàn chỉnh: người dùng nói câu hỏi dinh dưỡng, hệ thống nhận dạng transcript, truy xuất tri thức, sinh câu trả lời dạng streaming, chuyển từng đoạn sang TTS và phát âm thanh phản hồi trên trình duyệt. Kết quả này là cơ sở để chuyển sang phần thực nghiệm và đánh giá độ trễ, chất lượng RAG cũng như vai trò của từng mô-đun trong pipeline.

Các tham số đáng chú ý trong hệ thống gồm:

Bảng 4.1: Một số tham số cấu hình chính

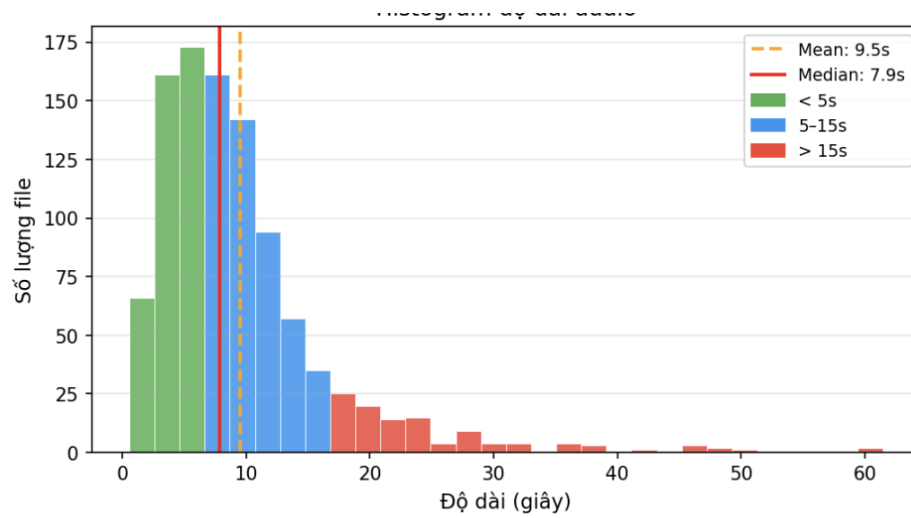
Tham số	Giá trị mặc định	Ý nghĩa
ASR_NUM_THREADS	4	Số luồng CPU cho Sherpa-ONNX.
MIN_TEXT_CHUNK_SIZE	40	Kích thước tối thiểu của text chunk trước khi gửi TTS.
Số chunk ứng viên từ Qdrant	15	Số chunk được lấy ở bước truy xuất ban đầu trước rerank.
Số chunk đưa vào LLM	5	Số chunk tốt nhất sau rerank được đưa vào đầu vào của LLM.
LLM_BACKEND	gemini	Backend sinh câu trả lời, có thể đổi sang OpenAI-compatible.
TTS_DEVICE	auto	Thiết bị chạy TTS, tự chọn CPU hoặc CUDA tùy môi trường.

4.3 Thiết lập thực nghiệm

Các thực nghiệm được thiết kế nhằm đánh giá hai khía cạnh chính của hệ thống: chất lượng câu trả lời dựa trên RAG và độ trễ trong tương tác giọng nói. Hệ thống được chạy trong môi trường Docker để tái lập cấu hình giữa các lần đo. Các thành phần Gateway, ASR, LLM/RAG, TTS, Qdrant, Redis và Web được khởi động như các service độc lập, trong đó các mô hình AI có thể sử dụng tăng tốc phần cứng bằng CUDA/GPU khi môi trường hỗ trợ.

Với đánh giá chất lượng RAG, mỗi mẫu đánh giá gồm câu hỏi, câu trả lời sinh bởi hệ thống và các ngữ cảnh được truy xuất từ kho tri thức. Các câu hỏi được chọn xoay quanh các chủ đề dinh dưỡng phổ biến như chế độ ăn cho bệnh nền, vitamin, khoáng chất, giảm cân, dinh dưỡng

cho trẻ em và phụ nữ mang thai. Với đánh giá độ trễ, hệ thống được đo trên nhiều cấu hình chunking khác nhau để phân tích ảnh hưởng của kích thước đoạn văn bản đến Time-to-First-Audio và tổng thời gian phản hồi.



Hình 4.1: Phân bố độ dài audio trong tập đánh giá

Hình trên cho thấy phần lớn các mẫu audio trong tập đánh giá có độ dài ngắn đến trung bình, tập trung chủ yếu quanh vùng dưới 15 giây. Giá trị trung bình khoảng 9.56 giây và trung vị khoảng 7.95 giây, cho thấy tập đánh giá phản ánh khá đúng kiểu câu hỏi thoại ngắn trong một hệ thống tư vấn. Tuy nhiên, phân bố vẫn có một số mẫu dài hơn 15 giây, thậm chí kéo dài tới khoảng một phút. Nhóm mẫu dài này quan trọng vì chúng giúp kiểm tra khả năng xử lý của ASR và pipeline streaming trong các trường hợp người dùng nói dài, đặt câu hỏi phức tạp hoặc mô tả nhiều điều kiện sức khỏe cùng lúc.

4.4 Đánh giá chất lượng RAG

Chất lượng của hệ thống được đánh giá bằng RAGAS, tập trung vào quan hệ giữa câu hỏi, câu trả lời và ngữ cảnh truy xuất. Các chỉ số quan trọng gồm:

- **Faithfulness:** mức độ câu trả lời bám sát ngữ cảnh được truy xuất.

- **Context Recall:** khả năng truy xuất đúng tài liệu cần thiết cho câu hỏi.
- **Answer Relevancy:** mức độ câu trả lời tập trung vào câu hỏi của người dùng.

Framework đánh giá chất lượng được xây dựng quanh RAGAS. Bộ dữ liệu đánh giá gồm câu hỏi, câu trả lời tham chiếu và ngữ cảnh liên quan. Với mỗi mẫu, hệ thống sinh câu trả lời và lưu lại context được truy xuất. Sau đó, RAGAS tính các chỉ số như faithfulness, answer relevancy, context precision và context recall. Cách đánh giá này phù hợp với hệ thống RAG vì nó không chỉ kiểm tra câu trả lời cuối cùng mà còn đánh giá chất lượng truy xuất ngữ cảnh.

Bảng dưới đây trình bày kết quả đánh giá RAGAS trên 360 mẫu với mô hình đánh giá Qwen/Qwen3.5-27B. Ba cấu hình được so sánh theo số lượng chunk ứng viên ban đầu và số chunk tốt nhất được đưa vào đầu vào của LLM.

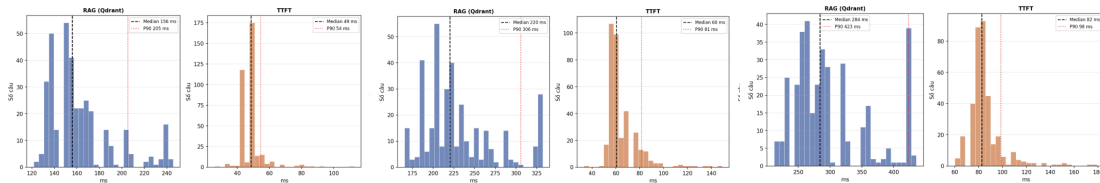
Bảng 4.2: Kết quả đánh giá chất lượng RAG theo cấu hình truy xuất

Cấu hình truy xuất	Answer Correctness	Faithfulness	Context Recall	Context Precision
10 chunk ứng viên, chọn 3 chunk	0.5887	0.7375	0.5184	0.5426
15 chunk ứng viên, chọn 5 chunk	0.6098	0.7723	0.6239	0.5709
20 chunk ứng viên, chọn 7 chunk	0.6136	0.7897	0.6830	0.5493

Kết quả cho thấy khi tăng số lượng chunk ứng viên và số chunk được đưa vào LLM, context recall và faithfulness có xu hướng tăng. Tuy nhiên, context precision không tăng đơn điệu: cấu hình 20 chunk ứng viên, chọn 7 chunk có recall cao hơn nhưng precision thấp hơn cấu hình 15 chunk ứng viên, chọn 5 chunk. Điều này cho thấy việc đưa thêm ngữ cảnh không luôn làm câu trả lời tốt hơn; hệ thống cần cân bằng giữa độ phủ

tài liệu, độ nhiều của context và chi phí latency. Trong phạm vi đồ án, cấu hình 15 chunk ứng viên, chọn 5 chunk được xem là điểm cân bằng hợp lý giữa chất lượng truy xuất và độ trễ cho voicebot.

Bên cạnh RAGAS, đồ án sử dụng ablation study để phân tích vai trò của từng thành phần trong pipeline. Cụ thể, hệ thống có thể so sánh kết quả khi có và không có reranker để đo ảnh hưởng của reranking đến độ phù hợp của ngữ cảnh; so sánh khi có và không có query expansion để đánh giá tác động của chuẩn hóa thuật ngữ đến độ phủ tri thức; đồng thời thay đổi kích thước chunk để phân tích trade-off giữa độ trễ, chất lượng giọng nói và tính đầy đủ của câu trả lời.



Hình 4.2: Phân bố thời gian RAG và thời gian sinh phần trả lời đầu tiên theo các cấu hình truy xuất

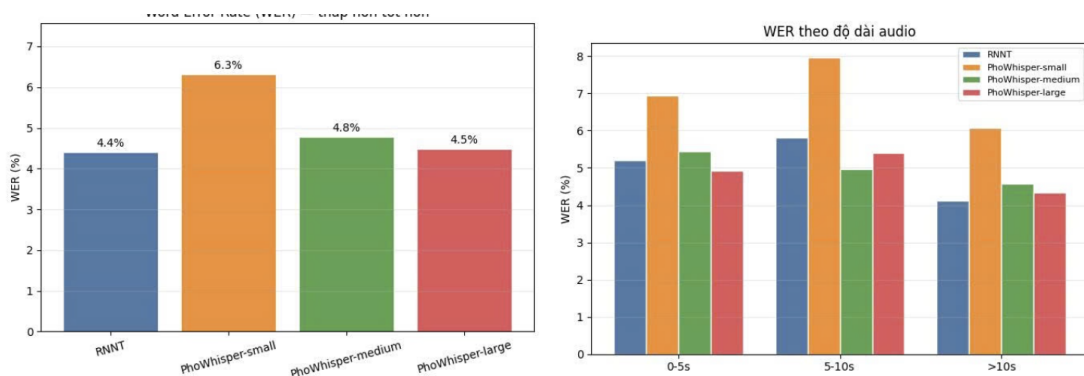
Ba biểu đồ trên minh họa ảnh hưởng của cấu hình truy xuất đến thời gian xử lý của khối LLM/RAG. Mỗi cấu hình gồm hai nhóm thời gian chính: thời gian RAG, tức thời gian mở rộng truy vấn, truy xuất và reranking; và thời gian sinh phần trả lời đầu tiên của LLM. Khi tăng số lượng chunk ứng viên và số chunk đưa vào LLM, thời gian RAG có xu hướng tăng vì hệ thống phải xử lý nhiều ứng viên hơn ở bước reranking và đưa nhiều ngữ cảnh hơn vào prompt. Ngược lại, thời gian sinh phần trả lời đầu tiên của LLM biến động ít hơn, do phần này phụ thuộc nhiều vào backend LLM và độ dài prompt sau khi đã chọn context.

Các biểu đồ này giúp giải thích lý do đồ án không chỉ chọn cấu hình có nhiều tài liệu nhất. Cấu hình truy xuất lớn hơn có thể cải thiện độ phủ ngữ cảnh, nhưng cũng làm tăng độ trễ trước khi LLM bắt đầu sinh câu trả lời. Vì voicebot cần phản hồi nhanh, cấu hình truy xuất phải cân bằng giữa chất lượng RAG và độ trễ cảm nhận của người dùng.

Bên cạnh chất lượng RAG, đề án còn đánh giá các thành phần riêng lẻ. ASR có thể được đo bằng WER và CER khi có transcript chuẩn. TTS được đánh giá bằng real-time factor, thời gian tạo chunk đầu tiên và đánh giá chủ quan MOS. Pipeline tổng thể được đo bằng thời gian từ lúc kết thúc câu nói đến audio đầu tiên. Việc tách các chỉ số theo module giúp xác định thành phần nào là nút thắt trong từng cấu hình.

4.5 Đánh giá độ trễ

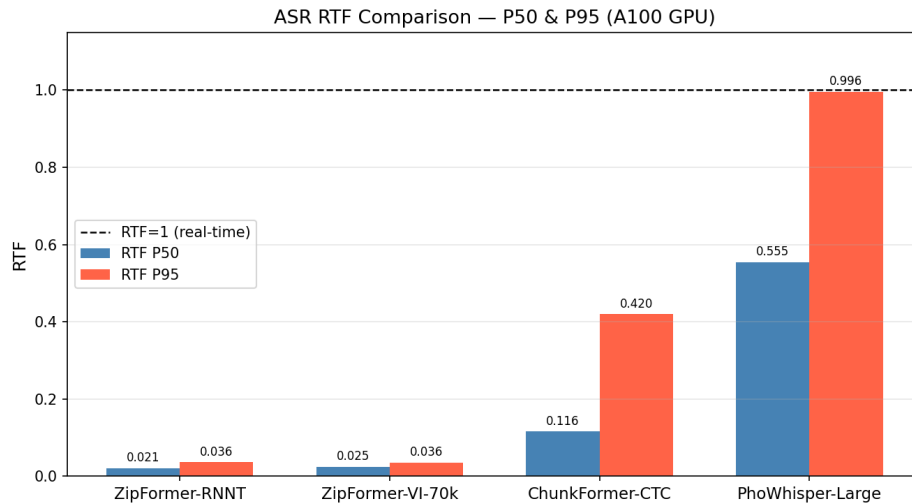
Độ trễ là tiêu chí cốt lõi của một hệ thống voicebot thời gian thực. Đề án tập trung đo các thành phần chính: thời gian ASR sinh transcript, thời gian RAG truy xuất và rerank, thời gian LLM sinh token đầu tiên, thời gian TTS tạo audio đầu tiên và thời gian end-to-end từ lúc người dùng nói xong đến lúc nghe phản hồi.



Hình 4.3: So sánh WER của các mô hình ASR theo mô hình và theo độ dài audio

Hình trên so sánh lỗi nhận dạng tiếng nói của nhiều mô hình ASR trên tập đánh giá tiếng Việt. Kết quả tổng thể cho thấy RNNT đạt WER khoảng 4.4%, PhoWhisper-small khoảng 6.3%, PhoWhisper-medium khoảng 4.8% và PhoWhisper-large khoảng 4.5%. Khi phân tích theo độ dài audio, WER thay đổi giữa các nhóm 0–5 giây, 5–10 giây và trên 10 giây, cho thấy chất lượng ASR không chỉ phụ thuộc vào mô hình mà còn phụ thuộc vào đặc điểm câu nói. Điều này phản ánh đặc điểm quan trọng của pipeline voicebot: transcript đầu vào không phải lúc nào cũng hoàn

hảo, và lỗi ASR có thể lan truyền sang bước truy xuất tri thức. Vì vậy, hệ thống cần có các bước giảm tác động của lỗi nhận dạng, chẳng hạn chuẩn hóa thuật ngữ dinh dưỡng, query expansion và truy xuất dựa trên nhiều biến thể câu hỏi.



Hình 4.4: So sánh real-time factor của các mô hình ASR

Biểu đồ real-time factor cho thấy sự khác biệt rõ giữa các mô hình ASR về tốc độ xử lý. Đường tham chiếu RTF bằng 1 biểu thị ngưỡng real-time: nếu RTF nhỏ hơn 1, mô hình xử lý nhanh hơn thời lượng audio đầu vào. ZipFormer-RNNT và ZipFormer-VI-70k có RTF rất thấp ở cả p50 và p95, cho thấy khả năng xử lý nhanh và ổn định hơn cho bài toán hội thoại thời gian thực. ChunkFormer-CTC vẫn nằm dưới ngưỡng real-time nhưng có p95 cao hơn, nghĩa là trong một số trường hợp chậm hệ thống có thể mất nhiều thời gian hơn. PhoWhisper-Large có p95 gần 1, tức là các trường hợp chậm gần sát ngưỡng real-time. Kết quả này cho thấy lựa chọn mô hình ASR ảnh hưởng trực tiếp đến độ trễ đầu vào của toàn bộ pipeline.

Các chỉ số được báo cáo theo percentile như p50, p90 và p99 để phản ánh cả trải nghiệm trung bình và trải nghiệm trong các trường hợp chậm. Trong đó, Time-to-First-Audio là chỉ số đặc biệt quan trọng vì quyết định cảm nhận phản hồi tức thời của người dùng. Cơ chế streaming song song

giữa LLM và TTS được kỳ vọng giúp giảm đáng kể chỉ số này so với pipeline tuần tự.

Thí nghiệm độ trễ được thực hiện trên bốn cấu hình: không chunking, chunk 80, chunk 40 và chunk 20. Mỗi cấu hình gồm 250 lượt đo. Kết quả thống kê cho thấy chunking làm giảm mạnh Time-to-First-Audio so với pipeline full. Bảng dưới đây tóm tắt một số kết quả chính:

Bảng 4.3: Kết quả đánh giá độ trễ theo cấu hình chunk

Cấu hình	Số mẫu	TTFA mean	TTFA p50	TTFA p90
Full	250	1614.2 ms	1523.0 ms	1760.9 ms
Chunk 80	250	1059.5 ms	1055.1 ms	1179.4 ms
Chunk 40	250	748.8 ms	746.7 ms	821.6 ms
Chunk 20	250	603.9 ms	603.4 ms	660.0 ms

Kết quả này cho thấy chiến thuật streaming có tác động rõ rệt đến độ trễ cảm nhận. Cấu hình full có p50 khoảng 1523 ms, gần sát ngưỡng mục tiêu 1.5 giây và có p90 vượt ngưỡng. Khi giảm kích thước chunk xuống 40 ký tự, p50 TTFA giảm còn khoảng 747 ms và p90 còn khoảng 822 ms. Với chunk 20, TTFA tiếp tục giảm nhưng có nguy cơ làm giọng nói bị ngắt đoạn nhiều hơn. Vì vậy, chunk 40 là lựa chọn cân bằng hơn giữa độ trễ và độ tự nhiên của audio.

Ngoài TTFA, tổng thời gian xử lý cũng giảm khi dùng chunking. Trung bình cấu hình full cần khoảng 4333.5 ms, trong khi chunk 40 cần khoảng 1508.9 ms và chunk 20 cần khoảng 1258.8 ms. Điều này phản ánh lợi ích của việc chồng lấp thời gian sinh văn bản và thời gian tổng hợp giọng nói. Tuy nhiên, kết quả latency cần được phân tích cùng với đánh giá MOS hoặc nghe thử để tránh tối ưu quá mức theo số liệu mà làm giảm trải nghiệm âm thanh.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đồ án đã đề xuất và xây dựng một hệ thống tư vấn dinh dưỡng thời gian thực qua giọng nói dựa trên pipeline mô-đun, streaming bất đồng bộ và RAG. Hệ thống kết hợp các mô-đun ASR, LLM và TTS để tạo thành pipeline voice-to-voice hoàn chỉnh. Điểm trọng tâm của giải pháp là tối ưu Voice RAG dưới ràng buộc độ trễ: câu hỏi được kiểm tra an toàn và chuẩn hóa trước khi truy xuất, tài liệu được chọn theo mức phù hợp, câu trả lời được sinh dựa trên ngữ cảnh truy xuất, và luồng LLM–TTS được điều phối để giảm thời gian phản hồi âm thanh đầu tiên.

Các đóng góp chính của đồ án gồm: thiết kế pipeline voice-to-voice mô-đun cho voicebot dinh dưỡng tiếng Việt; tổ chức pipeline Voice RAG dưới ràng buộc độ trễ của kênh thoại; áp dụng query expansion, truy xuất tài liệu và reranking để cải thiện chất lượng bằng chứng; đề xuất chiến thuật word-safe chunking kết hợp streaming song song giữa LLM và TTS; bổ sung guardrail an toàn cho các tình huống nhạy cảm; xây dựng bộ tiêu chí đánh giá kết hợp giữa chất lượng RAG, Time-to-First-Audio và độ trễ end-to-end.

Về mặt hiện thực, đồ án đã xây dựng được mã nguồn có đầy đủ các lớp cần thiết cho hệ thống: frontend React để thu và phát audio; Gateway FastAPI để điều phối WebSocket; ASR worker cho nhận dạng tiếng nói; LLM worker cho RAG và streaming câu trả lời; TTS worker cho tổng hợp giọng nói; Docker Compose để triển khai nhiều dịch vụ; các script đánh giá RAGAS, ASR, TTS và latency. Các thành phần này tạo thành một nền tảng thử nghiệm hoàn chỉnh cho bài toán tối ưu voicebot theo cả chất lượng và độ trễ.

Kết quả đánh giá chunking cho thấy kiến trúc streaming song song

là hướng đi hiệu quả. Khi chuyển từ pipeline full sang chunk 40, p50 Time-to-First-Audio giảm từ khoảng 1523 ms xuống khoảng 747 ms. Điều này chứng minh rằng không cần thay đổi mô hình nền, chỉ cần tổ chức lại luồng xử lý, kiểm soát kích thước chunk và chồng lấp LLM với TTS cũng có thể cải thiện đáng kể trải nghiệm hội thoại.

5.2 Hạn chế

Hệ thống vẫn tồn tại một số hạn chế. Chất lượng nhận dạng giọng nói có thể giảm trong môi trường nhiễu hoặc khi người dùng nói quá nhanh. Kho tri thức dinh dưỡng cần tiếp tục được mở rộng và cập nhật để bao phủ nhiều tình huống hơn. Ngoài ra, pipeline nhiều mô-đun tuy linh hoạt nhưng vẫn có chi phí vận hành và độ phức tạp triển khai cao hơn so với một mô hình end-to-end.

Một hạn chế khác là chất lượng cuối cùng phụ thuộc mạnh vào dữ liệu corpus. Nếu câu hỏi nằm ngoài phạm vi các bài viết đã thu thập, RAG có thể truy xuất ngữ cảnh không đủ thông tin, khiến LLM phải dựa nhiều hơn vào kiến thức nền. Bên cạnh đó, các mô hình embedding và reranking có chi phí tính toán đáng kể, đặc biệt khi chạy CPU. TTS cũng có trade-off giữa thời gian tạo audio đầu tiên và độ mượt khi đọc: chunk nhỏ giúp phản hồi nhanh hơn nhưng có thể tạo cảm giác ngắt câu.

Trong triển khai hiện tại, một số tham số vẫn cần tuning theo môi trường chạy thực tế, ví dụ số chunk ứng viên lấy từ Qdrant, số chunk tốt nhất đưa vào LLM, kích thước chunk văn bản cho TTS, backend LLM, thiết bị chạy ASR và TTS, cũng như mức concurrency. Vì vậy, kết quả trên một máy cụ thể không nên được xem là kết quả tuyệt đối cho mọi môi trường triển khai.

5.3 Hướng phát triển

Trong tương lai, hệ thống có thể được phát triển theo các hướng sau:

- 1) Tiếp tục mở rộng kho tri thức với nhiều nguồn y tế và dinh dưỡng được kiểm chứng hơn.
- 2) Hoàn thiện query triage để nhận biết câu hỏi ngoài phạm vi dinh dưỡng, câu hỏi mơ hồ và các tình huống cần khuyến nghị gặp bác sĩ.
- 3) Mở rộng truy xuất bằng hybrid retrieval, context compression nhẹ và semantic cache cho các câu hỏi phổ biến.
- 4) Cải thiện khả năng xử lý đồng thời bằng cách đo real-time factor của TTS, giới hạn số yêu cầu tổng hợp và hủy phản hồi cũ theo session.
- 5) Cải thiện khả năng xử lý tiếng nói trong môi trường nhiễu bằng VAD và ASR streaming tốt hơn.
- 6) Cá nhân hóa tư vấn dựa trên lịch sử hội thoại, độ tuổi, bệnh nền và mục tiêu dinh dưỡng của người dùng.

Ngoài các hướng trên, hệ thống có thể được mở rộng theo hướng đánh giá liên tục trong quá trình vận hành. Mỗi phiên hội thoại có thể ghi lại transcript, context truy xuất, câu trả lời, thời gian từng module và phản hồi người dùng. Dữ liệu này giúp phát hiện các câu hỏi corpus gap, bổ sung tài liệu mới vào kho tri thức và cải thiện từ điển query expansion. Khi số lượng dữ liệu thực tế đủ lớn, hệ thống có thể chuyển từ tuning thủ công sang tối ưu dựa trên thống kê sử dụng.

Tài liệu tham khảo

- [1] Alex Graves, Santiago Fernández, Faustino Gomez, Jürgen Schmidhuber. Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks. ICML, 2006.
- [2] Alex Graves. Sequence Transduction with Recurrent Neural Networks. arXiv:1211.3711, 2012. <https://arxiv.org/abs/1211.3711>
- [3] Ashish Vaswani et al. Attention Is All You Need. NeurIPS, 2017. <https://arxiv.org/abs/1706.03762>
- [4] Patrick Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020. <https://arxiv.org/abs/2005.11401>
- [5] Nils Reimers, Iryna Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP-IJCNLP, 2019. <https://arxiv.org/abs/1908.10084>
- [6] Yu. A. Malkov, D. A. Yashunin. Efficient and Robust Approximate Nearest Neighbor Search using Hierarchical Navigable Small World Graphs. IEEE TPAMI, 2020. <https://arxiv.org/abs/1603.09320>
- [7] Qdrant Documentation. Indexing. <https://qdrant.tech/documentation/manage-data/indexing/>
- [8] Shahul Es, Jithin James, Luis Espinosa-Anke, Steven Schockaert. RAGAS: Automated Evaluation of Retrieval Augmented Generation. arXiv:2309.15217, 2023. <https://arxiv.org/abs/2309.15217>

- [9] Jaehyeon Kim, Jungil Kong, Juhee Son. Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech. ICML, 2021. <https://arxiv.org/abs/2106.06103>
- [10] Chengyi Wang et al. Neural Codec Language Models are Zero-Shot Text to Speech Synthesizers. arXiv:2301.02111, 2023. <https://arxiv.org/abs/2301.02111>
- [11] FastAPI Documentation. WebSockets. <https://fastapi.tiangolo.com/advanced/websockets/>